

Detecting and Mitigating OWASP Top 10 Threats: Design, Implementation and Security Evaluation of a Database-Driven Web App

Name: Zubair Idris Aweda

Student Number: 2306535

Location/Site: City Campus

Module: 6CS007, Project and Professionalism

Supervisor: Alix Bergeret

Reader: Sherin Nassa

Date: 24/04/2026

Award Title: BSc (Hons) Computer Science

Table Of Contents

Table Of Contents.....	1
1. Introduction.....	4
1.1 Project Motivation.....	4
1.2 Introduction to OWASP Top Ten.....	5
1.3 Aims and Objectives.....	6
1.3.1 Academic Question.....	6
1.3.2 Aims.....	6
1.3.3 Objectives.....	6
1.4 Scope and Limitations.....	7
1.5 Report Framework.....	7
1.6 Summary.....	8
2. Literature Review.....	9
2.1 Introduction to Web Application Security.....	9
2.1.1 The Role of Web Applications in Modern Society.....	9
2.1.2 The Importance of Secure Software Design.....	9
2.1.3 The Role of OWASP in Web Security.....	9
2.2 Evolution of Web Vulnerabilities.....	10
2.3 Identifying and Detecting Vulnerabilities.....	10
2.3.1 Manual Detection Methods.....	10
2.3.2 Automated Detection Approaches.....	11
2.3.2.1 Static Analysis (SAST).....	11
2.3.2.2 Dynamic Analysis (DAST).....	11
2.3.2.3 Hybrid and AI-Assisted Methods.....	11
2.4 The OWASP Top 10 Framework.....	11
2.4.1 Overview and Methodology.....	11
2.4.2 Detailed Overview of Each Category.....	12
2.4.2.1 Broken Access Control.....	12
2.4.2.2 Cryptographic Failures.....	12
2.4.2.3 Injection.....	12
2.4.2.4 Insecure Design.....	13
2.4.2.5 Security Misconfiguration.....	13
2.4.2.6 Vulnerable and Outdated Components.....	13
2.4.2.7 Identification and Authentication Failures.....	14
2.4.2.8 Software and Data Integrity Failures.....	14
2.4.2.9 Security Logging and Monitoring Failures.....	14
2.4.2.10 Server-Side Request Forgery (SSRF).....	14
2.5 Mitigating and Preventing OWASP Top 10 Vulnerabilities.....	15
2.5.1 Secure Development Practices.....	15
2.5.2 Technical Mitigation Strategies.....	15

2.5.3 Role of Automation and DevSecOps.....	15
2.5.4 Compliance and Governance Considerations.....	16
2.6 Human Factors and Security Awareness.....	16
2.6.1 Developer Knowledge and Perception of Risk.....	16
2.6.2 The Role of Education and Training.....	16
2.7 Evaluating Tools and Frameworks.....	17
2.7.1 Evaluation Criteria.....	17
2.7.2 Comparative Effectiveness of Manual vs Automated Testing.....	17
2.8 Gaps in Existing Research.....	17
2.9 Summary.....	18
3. Methodology and Artefact Design.....	20
3.1 Introduction.....	20
3.2 Design.....	20
3.2.1 Technology and System Design Choices.....	20
3.2.2 System architecture model.....	21
3.2.3 Database schema model.....	22
3.2.3.1 Relationships.....	22
3.3 Implementation.....	22
3.3.1 Vulnerability Introduction Strategy.....	23
3.3.2 Mitigation Implementation.....	24
3.3.3 Function Requirements.....	25
3.3.3.1 Mockups.....	26
3.3.4 Non-Functional Requirements.....	28
3.3.5 Privacy and Ethics.....	29
3.4 Summary.....	29
4. Testing and Evaluation.....	30
4.1 Introduction.....	30
4.2 Testing.....	30
4.2.1 Purpose of Testing.....	31
4.2.2 Testing Methods.....	31
4.2.2.1 Manual Testing.....	31
4.2.2.2 Automated Testing.....	32
4.2.2.3 Sequence Diagrams.....	33
4.3 Evaluation.....	36
4.3.1 Data Collection Types.....	38
4.4 Summary.....	38
5. Overall Results and Discussion.....	39
5.1 Introduction.....	39
5.2 Survey Results: Developer Knowledge and Attitudes.....	39
5.2.1 OWASP Awareness.....	39
5.2.2 Detection Methods and Tools.....	39

5.2.3 Confidence and Barriers.....	40
5.2.4 Prioritisation and Remediation Behaviour.....	40
5.2.5 Qualitative Findings.....	40
5.3 Automated Tool Results.....	41
5.3.1 SonarQube (SAST).....	41
5.3.2 OWASP ZAP (DAST).....	42
5.3.3 Snyc.....	43
5.4 Manual Testing Results.....	44
5.4.1 Key Findings by Vulnerability Category.....	47
5.4.1.1 SQL Injection (A03).....	47
5.4.1.2 Security Misconfiguration (A05).....	47
5.4.1.3 Insecure Design and Business Logic (A04).....	47
5.4.1.4 Broken Access Control and Predictable Identifiers (A01).....	48
5.4.1.5 Cryptographic Failures (A02).....	48
5.4.1.6 Testing Limitations and Blocked Tests.....	48
5.5 Comparative Analysis: Manual vs Automated Detection.....	48
5.5.1 Strengths and Limitations of Each Method.....	49
5.5.1.1 Manual Testing.....	49
5.5.1.2 SonarQube (SAST).....	50
5.5.1.3 OWASP ZAP (DAST).....	50
5.5.1.4 Snyc (SCA).....	51
5.5.2 Time and Effort Comparison.....	51
5.6 Addressing the Academic Question.....	51
5.7 Summary.....	52
6. Conclusion and Future Work.....	54
6.1 Summary of Findings.....	54
6.2 Contributions.....	54
6.3 Practical Recommendations.....	55
6.4 Future Work.....	56
6.6 Closing Remarks.....	56
7. References.....	58
8. Appendices.....	61
8.1 Initial Gantt Chart.....	61
8.2 Final Gantt Chart.....	62

1. Introduction

1.1 Project Motivation

Web applications are central to almost every aspect of modern life, powering services in finance, healthcare, education, and entertainment. However, their growing complexity and interconnectivity have made them key targets for cyberattacks. The Open Worldwide Application Security Project (OWASP) plays a crucial role in addressing these challenges by maintaining the OWASP Top 10, a regularly updated list of the most critical security risks affecting web applications (OWASP, 2021a; OWASP, 2021b).

Despite the accessibility of these guidelines, many web applications continue to exhibit preventable vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), and Broken Access Control. These recurring issues often stem from gaps in developer awareness, weak testing practices, and limited integration of security principles into the software development lifecycle (Shahid et al., 2022; Codacy, 2025). This project is therefore motivated by the need to bridge the gap between security theory and practical implementation, helping developers not only understand the OWASP Top 10 risks but also apply effective methods to detect and mitigate them in real-world systems.

The research will involve designing a deliberately vulnerable, database-driven web application that includes examples of common OWASP Top 10 vulnerabilities. The same application will then be redeveloped using best-practice mitigation strategies to create a secure version. Both versions will be evaluated through manual penetration testing and automated tools such as OWASP ZAP, Snyk, and SonarQube (Qadir et al., 2025; Koman & Janiszewski, 2025). This process will make it possible to compare detection accuracy, false positives, and overall effectiveness across different testing methods.

Studies have shown that static and dynamic analysis approaches each have strengths and weaknesses in identifying vulnerabilities (Ferrara et al., 2020; Koman & Janiszewski, 2025). For example, static analysis can reveal flaws in source code before deployment, while dynamic scanners detect runtime vulnerabilities that static tools may miss. However, even the best tools often struggle with complex attack scenarios such as broken access control (Anas et al., 2024) or cryptographic failures (Invicti, 2025). By applying both methods to a controlled web environment, this project will evaluate how automation and human expertise can be combined for more effective vulnerability management.

The ultimate goal of this research is not just to identify security flaws but to understand how they can be systematically prevented through design, testing, and awareness. The project will conclude with a comprehensive set of practical guidelines. A clear, actionable framework that developers and organisations can adopt to identify, prioritise, and mitigate OWASP Top 10 vulnerabilities efficiently. These guidelines will serve as a bridge between academic insight and

real-world application, promoting a culture of secure software engineering that is proactive rather than reactive.

1.2 Introduction to OWASP Top Ten

The Open Worldwide Application Security Project (OWASP) is a global non-profit foundation dedicated to improving the security of software. It operates as an open community that provides freely available resources such as tools, documentation, and training materials to help developers and organizations build more secure systems. OWASP's mission is to make software security visible, enabling individuals and enterprises to make informed decisions about risk. With hundreds of local chapters and thousands of volunteers, OWASP plays a central role in defining industry standards for web application security (OWASP, 2021a).

One of its most influential initiatives is the OWASP Top Ten, a regularly updated, community-driven awareness document that identifies the most critical security risks to web applications. The list is based on data from security experts, organizations, and research studies worldwide, combining real-world exploit data with expert consensus. It serves as both an educational resource and a baseline for developing secure software. The OWASP Top Ten is widely recognized as the standard framework for web application security, used by developers, security professionals, auditors, and compliance bodies alike (OWASP, 2021b).

The most recent edition of the OWASP Top Ten (2021) includes the following categories:

1. Broken Access Control: Failures that allow users to act outside their intended permissions, often leading to unauthorized access or privilege escalation.
2. Cryptographic Failures: Issues related to the improper use or absence of encryption, leading to exposure of sensitive data.
3. Injection: Occurs when untrusted data is interpreted as code or commands (e.g. SQL, NoSQL, OS, or LDAP injection).
4. Insecure Design: Flaws in architecture or logic that create systemic weaknesses before code is even written.
5. Security Misconfiguration: Insecure default configurations, open cloud storage, or unnecessary features that leave systems exposed.
6. Vulnerable and Outdated Components: Use of outdated software libraries or dependencies with known security flaws.
7. Identification and Authentication Failures: Weak authentication mechanisms, poor session management, or credential exposure.
8. Software and Data Integrity Failures: Risks caused by unverified code or updates, supply chain compromises, or tampering with data integrity.
9. Security Logging and Monitoring Failures: Insufficient monitoring or alerting mechanisms that prevent timely detection and response to breaches.
10. Server-Side Request Forgery (SSRF): Vulnerabilities that allow attackers to make requests from a vulnerable server, potentially accessing internal systems or cloud metadata.

This research uses the OWASP Top Ten as both a framework and a benchmark for evaluating the security posture of web applications. By implementing and testing each vulnerability type in a controlled environment, the project aims to produce practical insights and evidence-based recommendations for detecting, preventing, and mitigating these common risks.

1.3 Aims and Objectives

1.3.1 Academic Question

The academic question of this report will be the following:

How can OWASP Top 10 threats be best managed in modern database-driven web applications?

This research question will be addressed through the following aim and objectives, which are designed to comprehensively investigate the management of OWASP Top 10 threats in web applications.

1.3.2 Aims

- Investigate and demonstrate how OWASP Top 10 vulnerabilities manifest in modern web applications by designing a deliberately vulnerable system.
- Evaluate detection and mitigation approaches by comparing manual testing and automated tools, then implementing a secure version of the application.
- Analyse and provide recommendations on the effort, impact, and trade-offs of vulnerability management, offering best practices for improving security

1.3.3 Objectives

- Conduct a detailed literature review of the OWASP Top 10 vulnerabilities, their real-world impacts, severity, and relation to compliance frameworks.
- Investigate developer knowledge, awareness, and approaches to OWASP vulnerabilities through surveys or small usability tests, examining how awareness impacts detection and remediation.
- Develop a deliberately vulnerable database-driven web application that includes instances of OWASP Top 10 vulnerabilities for practical demonstration.
- Apply both manual testing techniques and automated tools (e.g., OWASP ZAP, Snyk, SonarQube, Dependency-Check) to identify vulnerabilities; compare detection accuracy, time, and false positives.
- Implement a secure version of the same application, applying best-practice coding patterns and mitigations for each OWASP category.
- Collect data on detection rates, remediation times, complexity of fixes, and impact reduction before and after mitigation.
- Produce practical guidelines and recommendations for developers, educators, and organisations on best practices for managing OWASP Top 10 threats.

1.4 Scope and Limitations

The scope of the project is to evaluate methods for detecting and mitigating web application vulnerabilities using the OWASP Top 10 framework. A database-driven web application will be developed as the main artefact to demonstrate and assess practical techniques for identifying and resolving common security flaws.

The project will not attempt to cover all possible web security risks or extend to mobile, IoT, or other API-specific vulnerabilities. Its focus will remain strictly on the OWASP Top 10 framework to ensure depth of analysis and practical demonstration within a manageable scope.

A key limitation is time. The development, testing, and evaluation phases must be completed within one academic semester, which restricts how many tools, configurations, or testing cycles can be explored in detail.

Another limitation relates to resources and tools. Some security testing frameworks may require premium features or advanced infrastructure, potentially limiting access to certain capabilities or large-scale testing environments.

Human participation, such as surveys or usability testing, may also be constrained by low response rates or participant availability, which could affect the breadth of data collected.

Finally, while the project aims to provide comprehensive guidelines for identifying and mitigating OWASP Top 10 vulnerabilities, the findings will be based on a controlled environment and may not fully represent the complexity of large-scale, real-world systems.

1.5 Report Framework

The final report will be structured into six distinct chapters, logically progressing from the theoretical foundation to the practical implementation and evaluation of the project. The framework is designed to address the academic question regarding the management of OWASP Top 10 threats and fulfill the project aims.

Chapter 1 establishes the context of the research, highlighting the ubiquity of web applications and the growing security risks they face. It defines the project's motivation, specifically the need to bridge the gap between theoretical security guidelines and practical developer implementation. The chapter also outlines the specific aims and objectives, the academic question, and the scope and limitations of the study.

Chapter 2 provides a critical analysis of existing academic and industry literature. It examines the evolution of web vulnerabilities, the role of the OWASP Top 10 framework, and the strengths and weaknesses of current detection methodologies, including Manual Penetration Testing, Static Application Security Testing (SAST), and Dynamic Application Security Testing (DAST). It also identifies gaps in current research, specifically the lack of integrated studies comparing these methods within a controlled environment.

Chapter 3 details the research methodology and the technical development of the primary artefact: a database-driven web application. It describes the design and implementation of two distinct versions of the application, a deliberately vulnerable version containing controlled instances of OWASP Top 10 flaws, and a hardened, secure version . This section justifies the technology stack, the selection of specific vulnerabilities, and the testing protocols used for evaluation.

Chapter 4 presents the empirical data collected during the testing phase. It documents the results of applying manual penetration testing and automated tools (such as OWASP ZAP, Snyk, and SonarQube) to both versions of the application. The analysis focuses on key metrics defined in the project objectives, including detection rates, false positives, time-to-detect, and the complexity of required fixes.

Chapter 5 interprets the findings from the evaluation phase, discussing the comparative effectiveness of manual versus automated testing. Drawing on these results, it presents the practical guidelines and recommendations for developers and organisations. This section addresses the trade-offs involved in vulnerability management and offers best practices for embedding security into the development lifecycle.

Chapter 6 summarises the research findings in relation to the initial academic question. It reflects on the project's contributions to the field of web application security, acknowledges the limitations encountered during the study, and proposes areas for future research to further advance the understanding of automated and manual vulnerability management.

1.6 Summary

This chapter presented an introduction to the context of web application security and the pivotal role of the OWASP Top 10 framework in addressing modern cyber threats . It highlighted the persistence of preventable vulnerabilities caused by gaps in developer awareness and testing practices, establishing the project's motivation to bridge the divide between security theory and practical implementation through the development of a deliberately vulnerable and hardened web application.

The academic question, aims, and objectives for the project were defined and will be used to structure the investigation into the comparative effectiveness of manual penetration testing versus automated security tools . These elements, alongside the defined scope and limitations, serve to focus the research on evaluating vulnerability management strategies within a controlled environment.

Chapter 2 will be the literature review process that will give an overall view of the evolution of web vulnerabilities and detection methodologies, identifying gaps in existing research to justify the technical approach selected for this project.

2. Literature Review

2.1 Introduction to Web Application Security

2.1.1 The Role of Web Applications in Modern Society

Web applications now underpin several essential services across finance, healthcare, government, entertainment, and education. Their shift from static, brochure-style websites to complex, interactive systems has rapidly expanded their attack surface. Modern applications integrate third-party libraries, cloud infrastructure, APIs, microservices, and distributed authentication mechanisms, making them more interconnected. This new level of connection thus makes them more exposed (Stuttard & Pinto, 2011). As highlighted in recent literature, this dependency has created a scenario where even small implementation flaws can have system-wide consequences, often magnified by fast development cycles and continuous deployment practices (Khan et al., 2024).

Because of this, web application security has evolved from a niche technical concern to a fundamental requirement for organisational resilience.

2.1.2 The Importance of Secure Software Design

Secure software design emphasises preventing vulnerabilities before they emerge, rather than merely detecting them afterwards. Research shows that insecure design decisions, such as missing authentication boundaries, poor data validation pathways, and weak architectural assumptions, are root causes of many recurring vulnerabilities (Codacy, 2025; Wisse et al., 2023). Government guidance similarly stresses that secure-by-design practices form the foundation of long-term cyber resilience, as insecure systems cannot be effectively “patched into” safety once deployed (UK Government, 2022).

This perspective aligns with modern security engineering approaches, which place architectural modelling, threat analysis, and early-stage mitigation at the core of the software development life cycle (SDLC). Without secure-by-design principles, even advanced testing tools cannot fully mitigate systemic weaknesses.

2.1.3 The Role of OWASP in Web Security

The Open Web Application Security Project (OWASP) is a non-profit foundation that provides community-driven guidance, tools, and educational resources to improve web application security. Its OWASP Top 10 is widely regarded as a pragmatic, industry-informed summary of the most critical classes of web application risk; it combines empirical vulnerability data with expert input to prioritise common threats and recommended controls (OWASP, 2021). Because it is both accessible and actionable, the Top 10 serves as a reference point for developers, testers, and managers when organising security activities, from threat modelling to penetration testing and compliance mapping.

Beyond the Top 10, OWASP's role extends to toolkits, secure-coding guides, and training materials that aim to translate abstract security concepts into implementable practices. Academic and industry literature frequently uses OWASP as the baseline taxonomy for vulnerability studies and tool evaluations because it provides a shared vocabulary and a pragmatic set of priorities for remediation (OWASP, 2021). However, while OWASP effectively highlights prevalent risks, researchers note that organisations must adapt its guidance to their specific contexts. This means combining OWASP-led awareness with secure design, automated tooling, and governance practices to achieve sustained security improvements.

2.2 Evolution of Web Vulnerabilities

The earliest and most persistent web vulnerabilities have historically stemmed from input validation weaknesses, specifically SQL injection, Cross-Site Scripting (XSS), and command injection. These flaws arise from insufficient boundary controls in dynamic applications and remain pervasive despite decades of industry awareness. As noted in recent historical reviews, legacy threats such as SQL injection continue to be among the most widely exploited vulnerabilities due to their relative ease of execution and the lingering presence of insecure coding patterns in legacy systems (Shahid et al., 2022).

As modern applications have transitioned toward cloud-native architectures, containerisation, and distributed authentication, the threat landscape has shifted from isolated coding errors to systemic architectural risks. Research indicates that misconfigurations within complex ecosystems, such as Kubernetes and Azure Active Directory, have emerged as leading causes of breaches (Bufalino et al., 2025; Bu Haimed et al., 2023). Furthermore, the modern attack surface has expanded to include dependency supply chain compromises, API misuse, and Server-Side Request Forgery (SSRF), demonstrating that security now depends as heavily on infrastructure integrity and secure configuration as it does on application logic.

2.3 Identifying and Detecting Vulnerabilities

2.3.1 Manual Detection Methods

Manual detection methods such as penetration testing, code reviews, and threat modelling remain the most contextually accurate approach for security assessment. Studies show that manual reviews excel at identifying logical flaws, chaining vulnerabilities, and uncovering weaknesses that scanners frequently miss (Armando & Rosalina, 2023). However, this approach is significantly constrained by the need for specialised expertise, its time and labour intensity, and the inherent difficulty of scaling across large, complex systems. Despite these logistical challenges, manual testing remains indispensable for verifying high-severity issues, particularly Broken Access Control and complex business logic flaws that automated tools often overlook.

2.3.2 Automated Detection Approaches

2.3.2.1 Static Analysis (SAST)

SAST tools analyse source code without executing it and excel at detecting specific technical issues, including unsafe string handling, encryption misuse, known vulnerable functions, and injection pathways. Static analysis research demonstrates strong performance for these defined code patterns but notes weaker effectiveness regarding contextual or environment-specific flaws (Ferrara et al., 2020). While tools like SonarQube and Snyk effectively leverage pattern matching and data-flow analysis, they frequently suffer from false positives due to this lack of runtime context.

2.3.2.2 Dynamic Analysis (DAST)

DAST tools analyse running applications by probing endpoints and behavioural responses to identify security weaknesses in real-time. Comparative studies (Koman & Janiszewski, 2025; Shahid et al., 2022) have found that scanners vary widely in terms of detection accuracy, coverage, and false positive rates, as well as their ability to identify complex multi-step attacks. For instance, while tools like OWASP ZAP perform strongly in basic injection detection, they often struggle with verifying access control or detecting stateful session vulnerabilities.

2.3.2.3 Hybrid and AI-Assisted Methods

Recent work has explored advanced detection techniques that combine static and dynamic analysis or incorporate machine-learning models to overcome the limitations of traditional tools. These approaches aim to improve accuracy in areas where existing scanners struggle, such as analysing complex data flows or reducing false positives. Examples include behavioural modelling for identifying broken access control patterns (Anas et al., 2024) and machine-learning methods used to detect SQL injection characteristics in network flow data (Crespo-Martínez et al., 2023).

More recent developments integrate large language models to support tasks such as inter-procedural taint analysis for identifying SSRF vulnerabilities in PHP systems (Ji et al., 2025). While these hybrid and AI-assisted approaches show promise, the literature indicates that they are still emerging, require significant computational resources, and are not yet widely embedded in mainstream development workflows. Nonetheless, they represent an important direction for improving the precision and scalability of vulnerability detection.

2.4 The OWASP Top 10 Framework

2.4.1 Overview and Methodology

The OWASP Top 10 remains the most influential classification of web application risks, serving as a standard awareness document for developers and web application security. OWASP compiles this list through broad industry surveys, empirical vulnerability data, and expert consensus. Each category represents families of related vulnerabilities rather than single

weaknesses, a structure that allows organisations to prioritise security resources based on real-world prevalence (OWASP, 2021).

Recent iterations of the framework have introduced notable changes that reflect shifts in the global threat landscape. Most significantly, Broken Access Control has risen to the number one position, while other risks have been broadly reframed, such as merging multiple attack classes under "Injection." Furthermore, the introduction of "Insecure Design" highlights the growing importance of architectural risk, while the recognition of "Software Integrity Failures" addresses escalating supply chain concerns. This evolution signals the critical need for developers to look beyond isolated coding vulnerabilities and understand the broader architectural, ecosystem, and behavioural security factors that define modern application defense.

2.4.2 Detailed Overview of Each Category

2.4.2.1 Broken Access Control

Broken Access Control remains one of the most prevalent and high-impact vulnerabilities in modern applications, occurring when systems fail to properly enforce permissions on resources or actions. Research highlights that access control flaws often stem from inconsistent rule enforcement, insecure direct object references, and predictable URLs that allow attackers to escalate privileges or retrieve protected data (Bright Security, 2023). More advanced studies demonstrate that broken access control is increasingly exploited through automated enumeration techniques and privilege-escalation chains, particularly in large, role-based systems (Anas et al., 2024). Because these weaknesses frequently arise from logic errors rather than code-level defects, they are difficult for automated tools to detect reliably, reinforcing the need for threat modelling and rigorous manual review.

2.4.2.2 Cryptographic Failures

Cryptographic failures (formerly "Sensitive Data Exposure") refer to weaknesses in how applications protect data both in transit and at rest. These include incorrect algorithm selection, poor key management, insecure protocol configurations, and failure to enforce encryption policies. Industry guidance shows that weak or outdated cryptography enables attackers to intercept, modify, or decrypt sensitive information, posing critical risks to confidentiality and integrity (Invicti, 2025). Academic work underscores that cryptographic failures are rarely caused by cryptography itself but by its misuse, such as default credentials, improper certificate validation, or insecure storage practices. As applications increasingly rely on distributed architectures and third-party services, cryptographic soundness has become central to ensuring trust across system boundaries.

2.4.2.3 Injection

Injection vulnerabilities occur when untrusted input is interpreted as part of a command or query, enabling attackers to manipulate execution flow. SQL Injection remains one of the most studied and dangerous forms due to its ability to compromise entire databases. Recent research shows that advanced detection techniques using machine learning can identify malicious traffic

patterns within network flows with over 97% accuracy, improving early detection of SQL-based exploitation attempts (Crespo-Martínez et al., 2023). Other studies propose holistic frameworks that combine CVSS/EPSS scoring with multi-source analysis to prioritise and prevent injection-related risks (Paul et al., 2024). Despite improved tooling, the persistence of injection vulnerabilities reflects fundamental challenges in input validation, sanitisation, and the misuse of unsafe APIs.

2.4.2.4 Insecure Design

Insecure Design refers to systemic weaknesses arising not from faulty code but from flawed architectural decisions, missing security controls, and inadequate threat modelling. Modern analyses emphasise that insecure design can manifest even in otherwise well-implemented code, highlighting the importance of embedding security considerations into the earliest stages of system design (Codacy, 2025). Poorly defined trust boundaries, incomplete security requirements, and missing workflow protections are recurring sources of design-level vulnerabilities. Because these threats cannot be resolved through patches alone, they require organisational investment in secure-by-design culture, architectural reviews, and continuous risk assessment throughout the development lifecycle.

2.4.2.5 Security Misconfiguration

Security misconfiguration remains a broad yet highly impactful category covering improper server settings, unnecessary features, overly permissive cloud configurations, and outdated components. Research shows that misconfigurations often serve as the initial foothold for attackers attempting lateral movement or privilege escalation, particularly in complex systems such as Kubernetes clusters (Bufalino et al., 2025) and identity platforms like Azure Active Directory (Bu Haimed et al., 2023). These studies demonstrate that modern misconfigurations frequently arise from the increasing complexity of infrastructure rather than simple human error. This makes automated configuration scanning, adherence to baseline standards, and continuous monitoring essential for reducing risk.

2.4.2.6 Vulnerable and Outdated Components

This category focuses on risks introduced by using outdated frameworks, libraries, or dependencies containing known vulnerabilities. The widespread adoption of open-source software has amplified the importance of tracking component health and applying timely patches. Invicti (2025) highlights that vulnerable components often remain exploitable due to poor inventory management and limited integration between software composition analysis and dynamic testing. The literature consistently notes that dependency vulnerabilities propagate quickly across ecosystems, requiring organisations to maintain rigorous update policies and implement automated dependency scanning to minimise exposure.

2.4.2.7 Identification and Authentication Failures

Identification and Authentication Failures refer to weaknesses in verifying user identities and enforcing robust authentication processes. Common issues include weak password policies, session fixation, insecure credential storage, and predictable recovery workflows. Cyolo (2025)

reports that authentication remains a major attack vector due to widespread reliance on outdated login flows and insufficient protection of credential-handling logic. Emerging research explores alternatives such as biometric-backed and blockchain-based authentication systems to address single-point-of-failure and transparency issues (Khalili et al., 2025). The persistence of authentication failures emphasises the need for multi-factor authentication, secure credential lifecycle management, and defence against automated credential-stuffing attacks.

2.4.2.8 Software and Data Integrity Failures

Software and Data Integrity Failures occur when applications fail to protect against tampering, unsafe updates, or compromised supply chains. Invicti (2025) notes that these vulnerabilities frequently arise in systems that implicitly trust external components, update mechanisms, or continuous deployment pipelines without verifying integrity. As software supply-chain attacks grow in frequency, researchers highlight the importance of cryptographic signing, checksum validation, and strict update policies to ensure that software and data remain authentic and unaltered. This category increasingly intersects with DevSecOps practices, requiring security controls to be embedded directly into CI/CD workflows.

2.4.2.9 Security Logging and Monitoring Failures

Security Logging and Monitoring Failures reflect an organisation's inability to detect, understand, or respond to malicious activity. Typical weaknesses include insufficient log detail, missing audit trails, poor log retention, and lack of alerting or correlation logic. ChaosSearch (2024) identifies recurring mistakes such as storing logs without context, failing to analyse them, or ignoring anomalies until after a breach occurs. The literature emphasises that effective monitoring is a prerequisite for incident response: without proper observability, even well-secured applications may remain compromised for long periods. As threats grow more sophisticated, real-time analytics and centralised log platforms have become increasingly important.

2.4.2.10 Server-Side Request Forgery (SSRF)

SSRF vulnerabilities allow attackers to trick servers into making unintended internal or external requests, often bypassing network protections. These attacks are particularly dangerous in cloud-hosted applications, where internal metadata services and private endpoints may be exposed if validation is incomplete. Recent research by Ji et al. (2025) demonstrates the use of LLM-assisted, path-sensitive taint analysis to improve SSRF detection in PHP applications, addressing a long-standing challenge in automated scanning. SSRF continues to gain prominence due to increased use of microservices and third-party APIs, making strict URL validation and network segmentation essential mitigations.

2.5 Mitigating and Preventing OWASP Top 10 Vulnerabilities

2.5.1 Secure Development Practices

Effective mitigation of OWASP Top 10 vulnerabilities begins with embedding security into the software development lifecycle. Secure development practices emphasise early threat

modelling, well-defined security requirements, and adopting secure-by-design principles that prevent architectural weaknesses before they manifest in code (Codacy, 2025). Research demonstrates that most high-impact vulnerabilities originate from design oversights, insufficient validation rules, and poorly defined trust boundaries. Integrating security considerations at the design stage, rather than relying on late-stage testing, reduces both technical debt and remediation cost. Frameworks and platforms should also enforce secure defaults, as recommended by Peguero and Cheng (2021), ensuring that developers are guided towards safer implementation patterns.

2.5.2 Technical Mitigation Strategies

Mitigating specific OWASP Top 10 vulnerabilities requires targeted technical controls tailored to each category. Broken Access Control can be addressed through systematic enforcement of role-based permissions and strict server-side checks (Anas et al., 2024), while cryptographic failures demand strong encryption policies, proper key management, and validated TLS configurations (Invicti, 2025). Injection attacks require robust input sanitisation, parameterised queries, and careful avoidance of dynamic interpreters (Crespo-Martínez et al., 2023). For security misconfigurations, continuous configuration auditing and baseline hardening are critical (Bufalino et al., 2025). Outdated component risks necessitate automated dependency tracking and reliable patching workflows, and integrity vulnerabilities require signature verification and tamper-resistant update mechanisms (Invicti, 2025). Across categories, literature consistently emphasises that technical mitigations are most effective when applied consistently and monitored for drift over time.

2.5.3 Role of Automation and DevSecOps

Automation plays an essential role in maintaining security at scale, particularly as modern systems become more distributed and complex. DevSecOps practices integrate security tooling, such as SAST, DAST, SCA, and IaC scanners, directly into build and deployment pipelines, ensuring that vulnerabilities are identified as early as possible (Koman & Janiszewski, 2025). The DevSecOps literature highlights that continuous automated scanning improves coverage and reduces manual workload while maintaining alignment with rapid release cycles (Prates & Pereira, 2025; Sehgal, 2023). However, automated tools still face limitations, including false positives and difficulty analysing design flaws, meaning that automated mechanisms must work alongside human oversight. The combination of automation, continuous integration, and secure deployment pipelines has become a cornerstone of modern mitigation strategies.

2.5.4 Compliance and Governance Considerations

Governance frameworks and compliance standards reinforce mitigation efforts by ensuring organisations implement minimum baseline protections. Guidance on security controls (Viegas & Kuyucu, 2022) and cyber-security compliance research (Wada, 2024) show that adherence to standards such as ISO 27001, NIST, or PCI-DSS improves consistency in access control, encryption, logging, and incident response. Compliance does not guarantee security but ensures accountability by mandating documented policies, secure update processes, and

monitored audit trails. Governance frameworks also help organisations justify investments in secure design, ensure appropriate separation of duties, and demonstrate due diligence in protecting user data.

2.6 Human Factors and Security Awareness

2.6.1 Developer Knowledge and Perception of Risk

Developer understanding of security concepts plays a decisive role in preventing OWASP Top 10 vulnerabilities. Research on human factors in web security shows that developers often underestimate the likelihood or impact of attacks, leading to incorrect assumptions about trust boundaries, inadequate validation, and misconfigured security controls (Pollini et al., 2022). These misunderstandings frequently arise from gaps in mental models. Developers may believe certain inputs are “safe,” assume frameworks handle security automatically, or overlook edge cases that attackers exploit. As a result, even well-designed systems can become vulnerable when secure practices are applied inconsistently or without full comprehension. Studies consistently highlight that improving developer perception of risk is essential for reducing recurring vulnerabilities, particularly in areas such as access control, injection, and authentication.

2.6.2 The Role of Education and Training

Targeted education and ongoing training are critical components for improving secure software development practices. Literature on security education (Sarker et al., 2024; Alyami et al., 2024) demonstrates that hands-on learning (through labs, capture-the-flag exercises, and guided code reviews) significantly enhances developers’ ability to detect and mitigate vulnerabilities. Training efforts that focus on real-world scenarios help bridge the gap between theoretical knowledge and practical implementation, reducing common mistakes introduced during coding or configuration. Regular upskilling is especially important in rapidly evolving threat landscapes, as outdated knowledge can lead to persistent vulnerabilities in production systems. By integrating structured training into development workflows, organisations can foster a culture of shared responsibility and continuous improvement in secure coding practices.

2.7 Evaluating Tools and Frameworks

2.7.1 Evaluation Criteria

Evaluating security testing tools requires assessing their accuracy, coverage, usability, and ability to detect vulnerabilities that align with the OWASP Top 10. Studies comparing scanning tools emphasise metrics such as true positive rate, false positive frequency, execution time, and ease of integration into development workflows as core indicators of effectiveness (Qadir et al., 2025; Koman & Janiszewski, 2025). Tools must also be evaluated for scalability, support for modern frameworks, and their ability to analyse both client-side and server-side logic. Together,

these criteria help determine how well a tool fits into real-world secure development practices and whether it provides reliable and actionable insights.

2.7.2 Comparative Effectiveness of Manual vs Automated Testing

Manual and automated testing provide complementary strengths, and the literature consistently highlights the importance of combining the two for comprehensive vulnerability detection. Automated methods such as SAST and DAST can quickly scan large codebases and identify common patterns of insecure behaviour, but they often struggle with contextual logic, complex authentication flows, and subtle business-logic flaws, leading to missed vulnerabilities or inflated false positives (Shahid et al., 2022; Qadir et al., 2025). Manual testing, particularly exploratory techniques and penetration testing, can uncover nuanced, application-specific weaknesses such as privilege escalation paths or insecure design flaws, but it is time-consuming and dependent on tester expertise. Research therefore supports hybrid approaches that integrate automated tooling with human-driven analysis to achieve greater depth and reliability in security assessment.

2.8 Gaps in Existing Research

Despite the breadth of existing work on web application security, the literature still lacks a single, comprehensive study that analyses all OWASP Top 10 categories together while also evaluating practical detection and mitigation techniques in an integrated, real-world context. Most research papers narrow their focus to individual vulnerabilities (such as SQL injection, broken access control, or SSRF) without offering a holistic view of how the full set of OWASP risks manifests within a modern web application. Similarly, comparative studies on security tools tend to evaluate scanners in isolation or under artificial benchmark conditions, leaving a gap in understanding how manual testing, SAST, DAST, and hybrid/AI-assisted methods perform when applied to the same system containing all OWASP Top 10 issues. This fragmentation makes it difficult for developers and organisations to rely on a single evidence-based source when deciding how to detect, prioritise, and remediate vulnerabilities.

Furthermore, existing research rarely links detection outcomes to practical mitigation steps in a structured way. Many studies demonstrate how a vulnerability can be identified or exploited, but fewer evaluate the effectiveness, effort, and trade-offs of specific fixes across different types of vulnerabilities. There is also limited examination of how developer knowledge, security culture, and real development workflows influence the success of vulnerability management strategies. These gaps highlight the need for a unified resource that not only explores each OWASP Top 10 threat in depth but also compares detection approaches side-by-side and provides clear, actionable guidance for building secure, modern applications. This project directly addresses this need by creating a deliberately vulnerable and secure web application, testing it systematically, and offering consolidated recommendations, positioning it as a potential go-to reference for practitioners and educators.

2.9 Summary

This chapter reviewed the evolution of web application security, highlighting how modern applications (shaped by cloud adoption, third-party dependencies, and continuous deployment) have expanded the attack surface of digital systems. As web applications increasingly underpin essential services, secure design has become a foundational requirement. The literature consistently emphasises that architectural decisions, not just coding mistakes, are responsible for many recurring vulnerabilities, reinforcing the importance of secure-by-design practices and early threat modelling. Within this landscape, OWASP plays a central role by providing a widely adopted taxonomy of the most critical risks and offering practical guidance, tools, and educational resources used by both practitioners and researchers.

Historical analyses show that while early vulnerabilities centred on input validation issues such as SQL injection and XSS, modern threats reflect more systemic weaknesses. These weaknesses include misconfigurations, supply-chain risks, complex authentication failures, and SSRF. The OWASP Top 10 updates mirror this shift, prioritising architectural and integrity-related vulnerabilities alongside traditional coding flaws. The chapter examined each OWASP Top 10 category in detail, drawing on academic and industry research to describe how these risks arise, how attackers exploit them, and which technical or design weaknesses contribute most to their persistence.

The chapter also explored current methods for identifying vulnerabilities. Manual approaches such as penetration testing remain essential for uncovering logic flaws and access control weaknesses, while automated tools (including SAST, DAST, and hybrid AI-assisted models) offer scalability and early detection but continue to struggle with contextual accuracy. Tool evaluations highlight significant variation in performance, reinforcing the value of combining manual and automated techniques. The chapter further discussed mitigation strategies spanning secure development practices, targeted technical controls, DevSecOps automation, and governance frameworks, as well as the human factors that influence secure coding behaviours. Research on developer mental models and training methods shows that knowledge gaps and misconceptions continue to contribute to preventable vulnerabilities, underscoring the need for continuous education.

Across the literature, several gaps remain. Most studies examine vulnerabilities or tools in isolation, lacking a unified analysis that spans all OWASP Top 10 categories and connects detection directly to mitigation in a real-world, controlled environment. Few works provide an integrated comparison of manual, automated, and hybrid methods applied to the same system. Likewise, practical, developer-oriented guidance consolidating effective detection and remediation techniques across all categories is still noticeably absent. These gaps justify the need for this project, which aims to create a deliberately vulnerable and secure web application, evaluate multiple detection methods side-by-side, and produce an accessible, evidence-backed set of recommendations that can serve as a comprehensive reference for managing OWASP Top 10 threats.

3. Methodology and Artefact Design

3.1 Introduction

This chapter outlines the methodology and design decisions underpinning the development of the project artefact. It explains how the system architecture, technologies, and implementation strategy were selected to support a controlled investigation of OWASP Top 10 vulnerabilities. The chapter also describes the A/B testing approach, in which two parallel versions of the same banking-style web application, one intentionally insecure and one securely engineered, are constructed to enable direct comparison of vulnerability behaviour and mitigation effectiveness. The following sections present the system architecture model, database schema, implementation approach, functional and non-functional requirements, and essential privacy and ethical considerations guiding the project.

3.2 Design

3.2.1 Technology and System Design Choices

The project will implement a small online banking-style web application, chosen because it is naturally database-driven and provides clear opportunities to explore OWASP-related risks in a controlled environment. Banking systems incorporate core features such as user authentication, session management, account viewing, money transfers, and transaction history, each of which maps directly to common security vulnerabilities including broken access control, injection risks, insecure session handling, and cryptographic failures. The application is sufficiently lightweight to be developed within the project timeline while still complex enough to demonstrate meaningful differences between secure and insecure system design.

To support the A/B testing methodology, two parallel versions of the application will be implemented. Version A will intentionally incorporate insecure design patterns, exposing typical weaknesses seen in real-world systems. Version B will implement the same functionality using secure coding practices and established mitigation strategies. This mirrored structure enables a direct, measurable comparison of how OWASP Top 10 vulnerabilities manifest and how they can be effectively prevented.

The application adopts a modern, API-driven architecture with a Node.js backend, a PostgreSQL database, and a Vue.js frontend. This design aligns with contemporary industry practice, offering clear separation of concerns and creating an ideal environment for security testing. By decoupling the frontend and backend, the system allows assessment of vulnerabilities along both the client-server boundary and internal backend logic, enabling more realistic evaluation of the OWASP Top 10 categories.

Node.js was selected for the backend because it is lightweight, widely adopted, and provides flexibility without enforcing strict security conventions by default. These characteristics make it

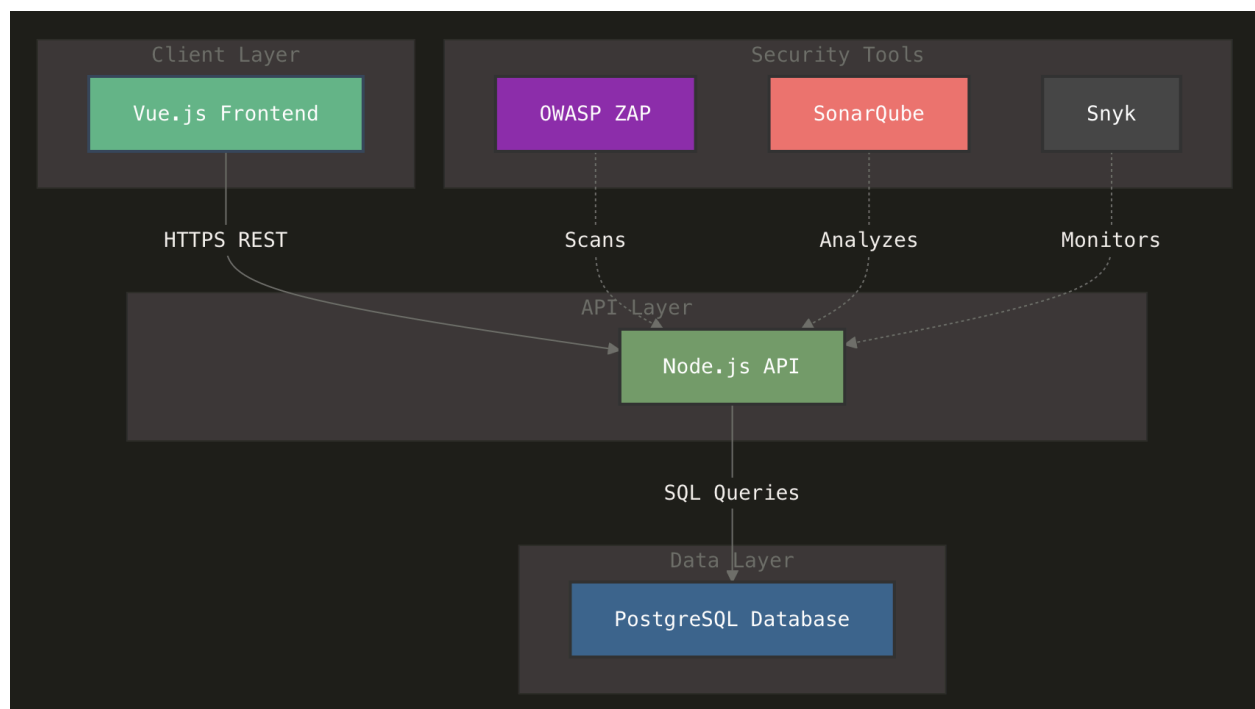
well-suited for building both a deliberately vulnerable version and a secure counterpart. Node.js also integrates well with static and dynamic analysis tools, dependency scanners, and runtime monitoring systems, all of which support the project's comparative testing objectives.

PostgreSQL was chosen as the database technology due to its reliability, lightweight configuration requirements, and straightforward deployment. Its strong SQL compliance allows the project to easily demonstrate injection vulnerabilities and secure alternatives such as parameterised queries. By minimising database administration overhead, PostgreSQL enables development efforts to remain focused on vulnerability evaluation rather than infrastructure complexity.

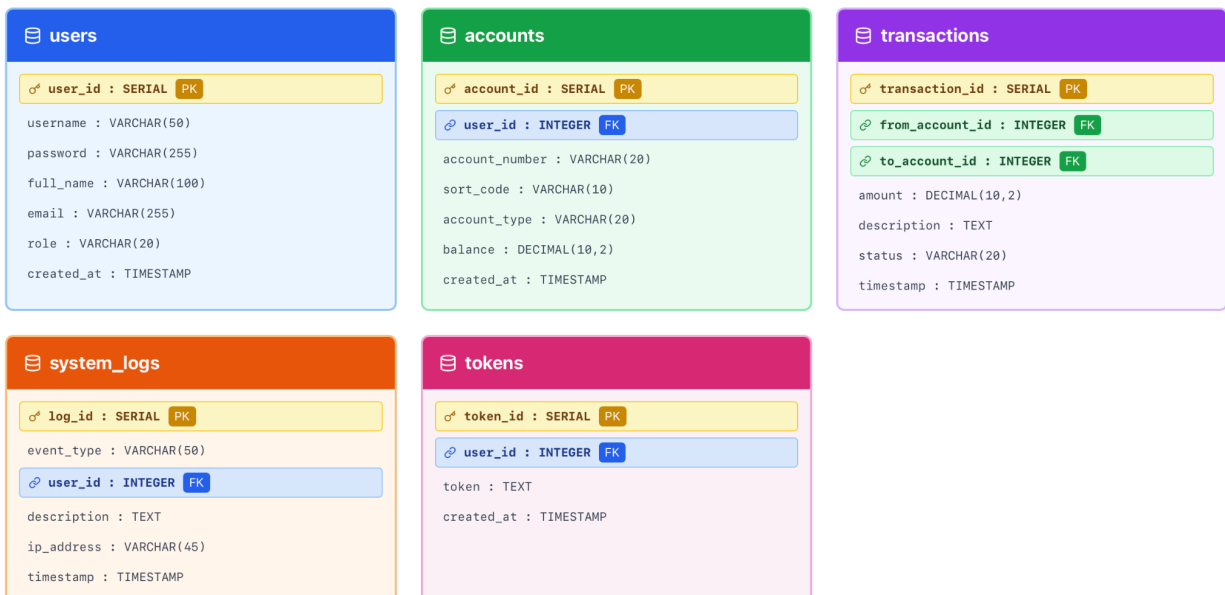
For the frontend, Vue.js provides an intuitive, component-based framework that supports rapid development and straightforward deployment. This ensures that the frontend does not become a bottleneck within the project while still offering opportunities to explore client-side risks such as insecure input handling, misconfigured authentication flows, or inadvertent exposure of sensitive data.

Overall, the API-centric architecture reflects the direction of modern web development, particularly in microservices and cloud-based environments. APIs expose distinct attack surfaces and allow clearer isolation of security concerns, making them ideal for studying OWASP-relevant threats. By combining this architecture with the A/B testing approach, the project creates a rigorous environment for examining vulnerabilities and validating the effectiveness of mitigation strategies.

3.2.2 System architecture model



3.2.3 Database schema model



3.2.3.1 Relationships

The database architecture is built around a series of one-to-many relationships that link the core Users entity to other critical components. Specifically, a single user is designed to be able hold multiple Accounts, allowing for distinct financial separations under one identity. Furthermore, each user is linked to multiple System Logs to maintain a comprehensive audit trail of their activities, as well as multiple Tokens, which supports the management of various active authentication sessions across different devices.

Financial activity is tracked through a direct relationship between accounts and transactions. In this structure, a single Account is associated with multiple transactions, creating a clear history of all funds moving in and out of that specific ledger.

To ensure data integrity and proper organization, the system utilizes standard key types and deletion rules. Primary Keys (PK) serve as unique identifiers for individual records, while Foreign Keys (FK) are employed to establish the links between these different tables. Additionally, a Cascade Delete rule is enforced for specific relationships; this means that if a user is deleted from the system, their associated accounts and tokens are automatically removed, preventing the accumulation of orphaned or inaccessible data.

3.3 Implementation

The implementation phase will involve building two parallel versions of the same banking web application so that direct comparisons can be made during testing. The first version will be deliberately vulnerable, incorporating insecure development practices to expose common

OWASP Top 10 risks. This version will use weak authentication mechanisms, such as plain session handling and minimal password requirements, and will rely heavily on client-side checks for access control. It will also make use of inline SQL queries without sanitisation, omit or improperly configure encryption, and display overly detailed error messages that may reveal system internals.

In contrast, the secure version of the application will follow industry best practices and secure-by-design principles. Authentication will implement hashed passwords, secure cookie handling, and the option for multi-factor authentication. Access control will be enforced strictly on the server side through role-based mechanisms. All database queries will be parameterised to prevent injection attacks, and encryption will be properly configured with TLS and correct key management procedures. Error handling will be sanitised to prevent the exposure of internal logic or sensitive data.

Both versions of the application will be structured as separate deliverables, organised into two folders, one containing the vulnerable build and the other containing the secure build. A detailed document will accompany them, outlining each vulnerability that was intentionally introduced into the insecure version, alongside the corresponding secure implementation used in the protected version. This dual-system setup provides a controlled environment for A/B testing, supporting the evaluation of detection methods and demonstrating the practical impact of secure development practices.

3.3.1 Vulnerability Introduction Strategy

Each OWASP Top 10 vulnerability category was introduced into Version A through deliberate, targeted implementation choices rather than accidental omission. SQL injection (A03) was introduced across all five backend controllers, `authController.js`, `accountController.js`, `transactionController.js`, `UserController.js`, and `authMiddleware.js`, by concatenating user-supplied input directly into SQL query strings using JavaScript template literals, for example constructing login queries of the form `SELECT * FROM users WHERE username = '${username}' AND password = '${password}'` rather than using parameterised placeholders. Cryptographic failures (A02) were introduced in two ways: passwords are stored as plaintext in the database with no hashing, and the JWT signing secret is hardcoded as the string `"super_secret_key_123"` directly in `authController.js` and committed to the public repository. Broken access control (A01) was introduced by trusting the user-supplied id URL parameter rather than the authenticated token identity when retrieving accounts and profiles, by allowing any authenticated user to update the role field on any account, and by using sequential integer primary keys (SERIAL) for all database tables, enabling trivial enumeration of user, account, and transaction records. Insecure design (A04) was introduced in the transfer endpoint by accepting and processing any numeric amount, including negative values, with no server-side validation that the amount must be positive, creating a business logic pathway for fraudulent self-credit. Security misconfiguration (A05) was introduced by configuring CORS with `origin: true` (a wildcard allowing any origin), by returning raw PostgreSQL error objects, including stack traces,

constraint names, and internal file references, directly in HTTP error responses, and by exposing all environment variables via the `/api` health-check endpoint using `env: process.env`. Vulnerable and outdated components (A06) were introduced by intentionally pinning `node-serialize` at version 0.0.4 in `package.json`, a package with a well-documented arbitrary code execution vulnerability. Identification and authentication failures (A07) were introduced by storing passwords in plaintext, by returning the full user object (including the password field) in registration and login responses, and by configuring JWT tokens with a 30-day expiry. Software and data integrity failures (A08) were introduced by serialising user preference cookies with `node-serialize` and deserialising them on every incoming request without any integrity verification, allowing a crafted cookie to execute arbitrary code on the server. Security logging and monitoring failures (A09) were introduced by omission: Version A contains no logging infrastructure whatsoever, no audit trail of authentication events, transfers, or errors is recorded anywhere. Finally, Server-Side Request Forgery (A10) was introduced in the avatar upload endpoint in `UserController.js`, where the server fetches an image by passing the user-supplied URL directly to `axios.get()` with no validation of the protocol, hostname, or whether the target is an internal network address.

3.3.2 Mitigation Implementation

Each vulnerability introduced into Version A was addressed in Version B through a corresponding secure implementation applied to the same endpoint or module. For SQL injection (A03), every instance of string concatenation was replaced with parameterised queries using PostgreSQL's numbered placeholder syntax, passing user-supplied values as a separate array so that input is never interpreted as executable SQL. For example, the login query in `authController.js` was rewritten from a template literal interpolating username and password directly into the query string, to a parameterised form accepting `$1` as the username placeholder and querying only by username before verifying the password separately using `bcrypt.compare()`. For cryptographic failures (A02), plaintext password storage was replaced with bcrypt hashing at a cost factor of 12 applied at registration and verified at login, and the hardcoded JWT secret was removed from source code entirely and loaded at startup from a `JWT_SECRET` environment variable, with the server refusing to start if the variable is absent. For broken access control (A01), all sequential integer primary keys (SERIAL) were replaced with UUID primary keys generated by PostgreSQL's `gen_random_uuid()` function across every table, and every protected endpoint was updated to compare the authenticated token identity (`req.user.userId`) against the requested resource owner before returning data, returning a 403 Forbidden response on mismatch. For insecure design (A04), the transfer endpoint was updated to reject any amount that is not a strictly positive number, enforced both through a Joi validation schema applied at the route level and through an explicit `transferAmount <= 0` guard in `transactionController.js`, ensuring negative and zero-value transfers are rejected before any database interaction occurs. For security misconfiguration (A05), the CORS configuration was updated to read the allowed origin from an environment variable rather than permitting any origin, raw database error objects were replaced with a centralised error-handling middleware that returns a single opaque message to

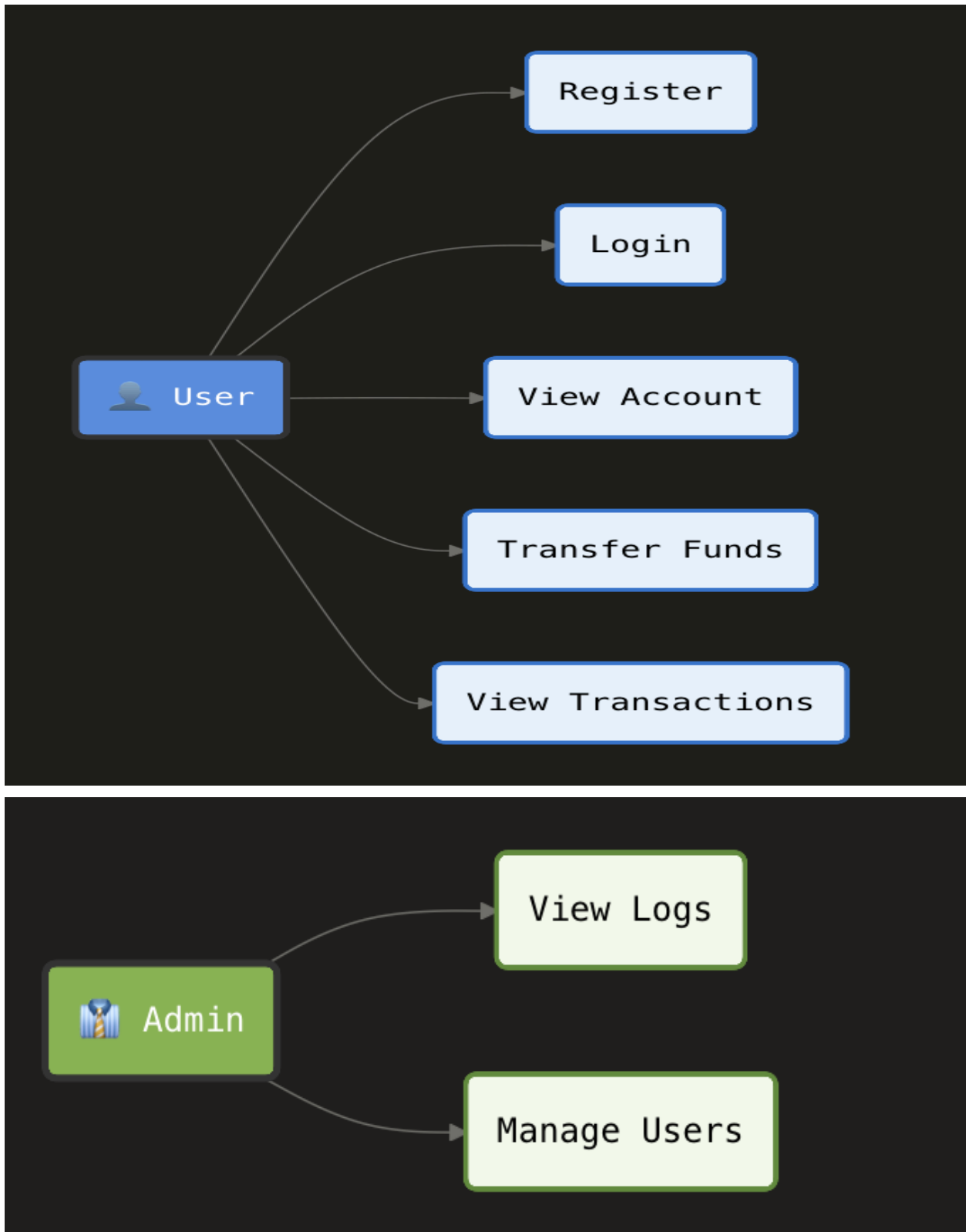
the client while logging the full error server-side, and the health-check endpoint was updated to expose only the application status and environment name rather than the full `process.env` object. For vulnerable and outdated components (A06), the `node-serialize` dependency was removed entirely from `package.json` and replaced with the native `JSON.parse` and `JSON.stringify` methods, which carry no known remote code execution risk. For identification and authentication failures (A07), password hashing with `bcrypt` replaced plaintext storage, API responses were updated to return only safe user fields (`user_id`, `username`, `full_name`, `email`, `role`) rather than the full database row, and the JWT expiry was reduced from 30 days to one hour. For software and data integrity failures (A08), the `node-serialize` cookie serialisation was replaced with `JSON.stringify`, the cookie was marked `httpOnly`, `signed`, and `sameSite: strict`, and the deserialisation middleware in `index.js` was updated to use `JSON.parse` on the signed cookie value rather than calling `unserialize()`, removing the arbitrary code execution pathway entirely. For security logging and monitoring failures (A09), a structured logging layer was introduced using `winston` and `winston-daily-rotate-file`, writing security-relevant events, including successful and failed authentication, transfer attempts, access control violations, and SSRF blocks, to both rotating log files and a `system_logs` database table, with a custom `PostgresTransport` class handling persistence. Finally, for Server-Side Request Forgery (A10), the avatar upload endpoint was updated to pass the user-supplied URL through an `isSafeUrl()` validation function before making any outbound request, rejecting any URL that does not use the https scheme or whose hostname matches a blocklist of private, loopback, and link-local address ranges, and validating that the response returns a permitted image content type before writing it to disk.

3.3.3 Function Requirements

The system must consist of two parallel versions of the same web application: a deliberately vulnerable implementation and a secure implementation. Both versions must provide identical functionality so that security differences can be evaluated accurately. At minimum, the application must support user registration, login, and role-based access control to allow broken access control and authentication weaknesses to be meaningfully tested. The system must also include basic data-handling operations, such as viewing, editing, and storing records, to enable evaluation of injection, cryptographic, and integrity-related risks. Input forms must accept user-supplied data, and the backend must process database queries, perform authentication checks, and return responses that testers can analyse.

The system must expose clearly defined routes, endpoints, and workflows so that automated scanning tools can crawl and analyse both implementations. It must also include server-side logic that can be examined for static analysis and for comparison between manual and automated detection methods. Finally, the artefact must generate sufficient logs and error responses to support later analysis during the evaluation stage.

3.3.3.1 Mockups



Login

Username	
●●●●●●●●	
Login	Register

Create Account

Full Name
Email
Username
●●●●●●●●
Register

User Dashboard

Account Balance £5,432.50	Account Details Account: ****1234 Sort Code: 12-34-56
Recent Transactions • Transfer to Account 5678 - £250.00 • Deposit - £1,000.00 • Payment to Merchant - £45.99 Claude content	
Transfer Funds	

Transfer Funds

⬆⬇⬆

Submit Transfer

Admin – System Logs

Timestamp	Event	User	IP	Description
2026-01-03 14:23:45	Login	john_doe	192.168.1.100	Successful login
2026-01-03 14:22:10	Transfer	alice_smith	192.168.1.105	Transfer £500 to account 5678
2026-01-03 14:20:33	Failed Login	unknown	10.0.0.50	Invalid credentials
2026-01-03 14:18:22	Registration	bob_jones	192.168.1.110	New user registered
2026-01-03 14:15:11	Transfer	carol_white	192.168.1.112	Transfer £250 to account 9012

3.3.4 Non-Functional Requirements

Both versions of the application must remain responsive under the load generated by automated scanning tools. OWASP ZAP's API scan targets all exposed endpoints in rapid succession, and the backend must handle this without crashing or producing misleading timeouts that could distort scan results. The Node.js and PostgreSQL stack was selected in part because its lightweight configuration supports this requirement without additional infrastructure.

Both versions must be deployable to a publicly accessible environment so that cloud-hosted scanning tools and remote testers can reach them under identical network conditions. Each version is deployed independently to Render, with its own PostgreSQL instance, ensuring that testing one version cannot affect the data or behaviour of the other.

The two codebases must remain structurally parallel throughout development. Controllers, routes, and database schema should mirror one another so that differences between versions

are attributable solely to security implementation choices rather than architectural divergence. This is enforced by keeping both projects under the same repository with matching folder structures.

The application must expose a complete Swagger API definition so that automated tools can discover and probe every endpoint without manual configuration. Both versions generate this definition automatically via `swagger-autogen` startup, ensuring the documentation remains accurate regardless of route changes.

Version B must enforce a minimum set of security behaviours independently of any specific vulnerability mitigation. These include refusing to start if critical environment variables (`JWT_SECRET`, `COOKIE_SECRET`) are absent, returning only opaque error messages to clients in all error paths, and logging all security-relevant events to both a rotating file and the `system_logs` database table. These requirements ensure the secure version represents a genuinely hardened system rather than one that merely patches individual vulnerabilities in isolation.

Neither version of the application may store or process real personal data at any point. All user accounts, credentials, and transaction records used during testing must be entirely fictitious. The deliberately vulnerable version must not be accessible beyond the controlled testing period to prevent unintended exploitation.

3.3.5 Privacy and Ethics

All testing activities must be conducted within a controlled environment using only the artefacts developed specifically for this project. No real user data, personal information, or external systems may be involved at any stage. The project must avoid scanning or interacting with any third-party websites, services, or networks to ensure that security testing remains legal and ethically sound. Any test accounts, credentials, or data samples used in the application must be entirely fictitious and generated solely for research purposes.

Ethically, the project must ensure that the deliberately vulnerable application cannot be deployed or accessed outside the testing environment, reducing the risk of real-world exploitation. All documentation of vulnerabilities must be presented responsibly, avoiding step-by-step exploit instructions that could be misused. The intention of the research is educational and evaluative, and all analysis must focus on improving security practices rather than demonstrating attack capability for its own sake. Finally, results must be reported in a way that respects academic integrity, acknowledges limitations, and avoids overstating the effectiveness of tools or methods.

3.4 Summary

This chapter has detailed the methodological foundations and design choices that guided construction of the project artefact. It justified the selection of a modern API-based architecture

using Node.js, PostgreSQL, and Vue.js, demonstrating how this combination supports realistic testing of OWASP Top 10 vulnerabilities. The A/B testing approach, comprising a deliberately vulnerable system and a securely engineered counterpart, provides a controlled environment for analysing how weaknesses emerge and how mitigation strategies perform in practice.

The chapter also outlined the implementation process, supported by system architecture modelling, database schema design, and the clear definition of functional and non-functional requirements. These ensured that both versions of the application remained structurally comparable and suitable for rigorous evaluation. Finally, the chapter established ethical safeguards, clarifying how vulnerabilities were introduced responsibly, how data privacy was maintained, and how testing remained confined to a safe, controlled environment. Together, these elements form the basis for the evaluation and analysis presented in subsequent chapters.

4. Testing and Evaluation

4.1 Introduction

This chapter outlines the testing and evaluation process used to assess both the vulnerable and secure versions of the web application. It explains how the A/B testing methodology was applied, describes the manual and automated techniques used to identify OWASP Top 10 vulnerabilities, and details how data was collected and interpreted. By deploying both systems under identical conditions and analysing their behaviour using OWASP ZAP, SonarQube, Snyk, and manual penetration testing, the chapter establishes a controlled foundation for comparing detection accuracy, identifying security weaknesses, and evaluating the practical impact of secure development practices.

4.2 Testing

The A/B testing approach in this project involves evaluating two versions of the same web application, one intentionally vulnerable and the other fully secured, to understand how different detection methods identify OWASP Top 10 threats across contrasting security postures. Version A represents the vulnerable implementation, and it is deployed first to establish a baseline for how many weaknesses can be discovered through manual testing and automated scanning. Once deployed, both manual testers and automated tools such as OWASP ZAP, SonarQube, and Snyk interact with the system in a controlled manner, and all findings are recorded, including vulnerability types, severity levels, and any issues that the tools fail to detect. This stage provides a comprehensive profile of the insecure system and highlights the kinds of flaws that are most easily exploited or overlooked.

After collecting results from Version A, Version B, the secure variant of the same application, is deployed using an identical environment and testing procedure. Maintaining consistent conditions ensures that any differences in test outcomes are attributable solely to the security

improvements made, rather than external variables. The same manual and automated tests are repeated to generate a comparable dataset, capturing detection rates, scanning behaviour, and performance across both versions. Once testing is complete, the results from both systems are analysed side-by-side. This comparison makes it possible to evaluate not only how effectively vulnerabilities were mitigated but also how accurately each testing method reflects real changes in security posture. The A/B test therefore provides an evidence-based understanding of the strengths and limitations of manual review, automated scanning, and hybrid approaches when applied to systems with clearly different levels of security.

4.2.1 Purpose of Testing

The purpose of the testing phase is to evaluate how the OWASP Top 10 vulnerabilities manifest within a real web application and to examine the effectiveness of different detection methods. By developing both a vulnerable and a secure version of the same system, the project enables a controlled comparison that shows how weaknesses emerge in practice and how those weaknesses can be identified.

Testing will also determine the relative strengths and limitations of manual testing techniques compared with automated security tools. Manual approaches, such as guided penetration testing and exploratory analysis, will be assessed for their ability to uncover logical or contextual flaws, while automated scanners will be evaluated for their ability to rapidly detect common patterns of insecure behaviour. This comparison provides insight into how each method contributes to a comprehensive security assessment.

Finally, the testing process will measure the improvements achieved through the mitigation strategies implemented in the secure version of the application. By analysing both environments side by side, the project will highlight the practical differences between insecure and secure design decisions. This A/B testing approach demonstrates not only how vulnerabilities can be detected, but also how effective remediation reduces risk, supporting the overarching goal of producing clear and actionable guidance for developers.

4.2.2 Testing Methods

4.2.2.1 Manual Testing

Manual testing focuses on exploring the vulnerable version of the web application through direct interaction, simulating the behaviours and thought processes of a real attacker. This involves attempting actions such as bypassing authentication mechanisms, tampering with user input, and accessing restricted areas of the application through forced browsing. The tester also conducts targeted SQL injection attempts and examines how the system responds to unexpected or malicious queries. Beyond technical exploits, manual testing also evaluates business logic weaknesses, such as determining whether a user can carry out actions, like transferring money or viewing another user's data, without the appropriate permissions.

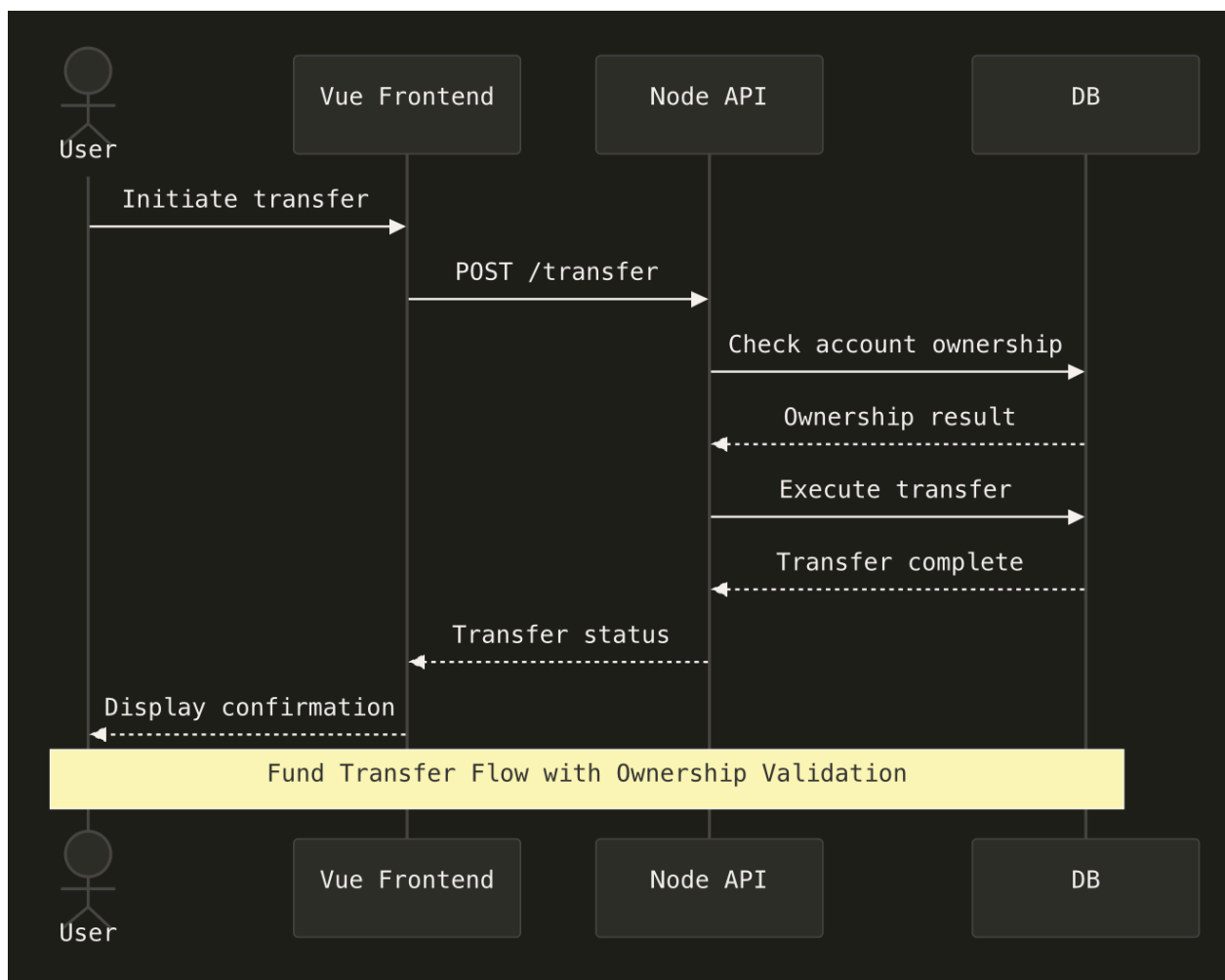
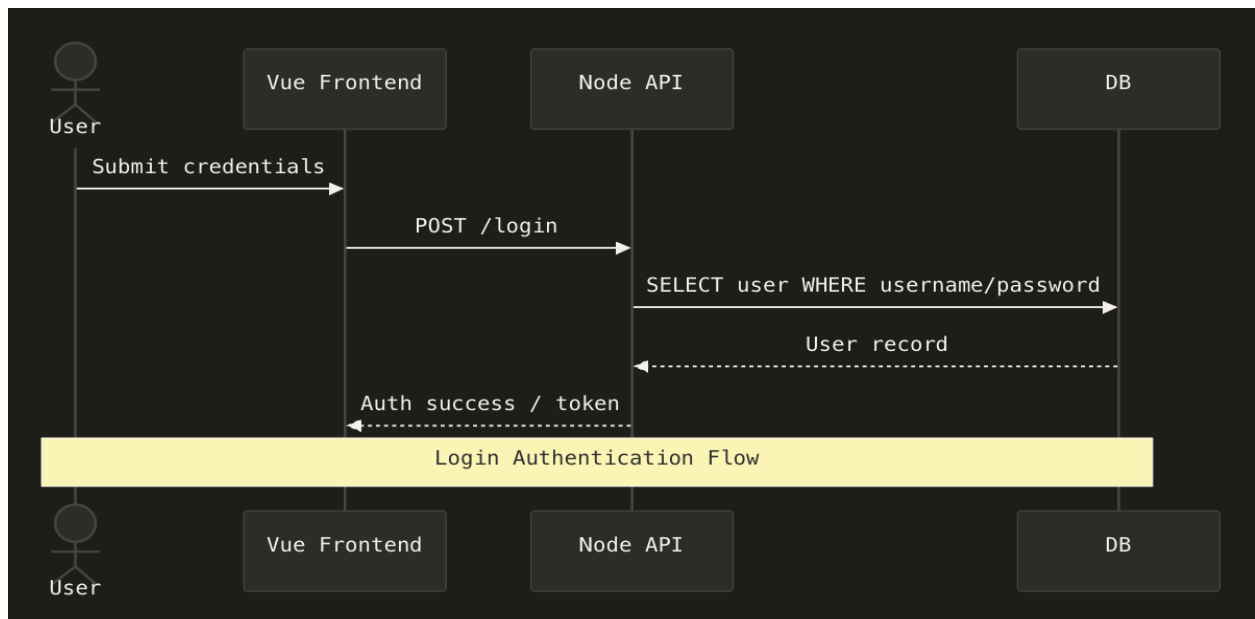
This approach is essential because it reveals vulnerabilities that automated scanners frequently overlook. Logical errors, broken workflows, and privilege-escalation paths often depend on contextual understanding rather than pattern matching, making them detectable only through careful, human-driven investigation. As highlighted by Armando and Rosalina (2023), manual methods remain one of the most effective ways to uncover complex or system-specific flaws, providing insights into how vulnerabilities can chain together in ways automated tools may never identify.

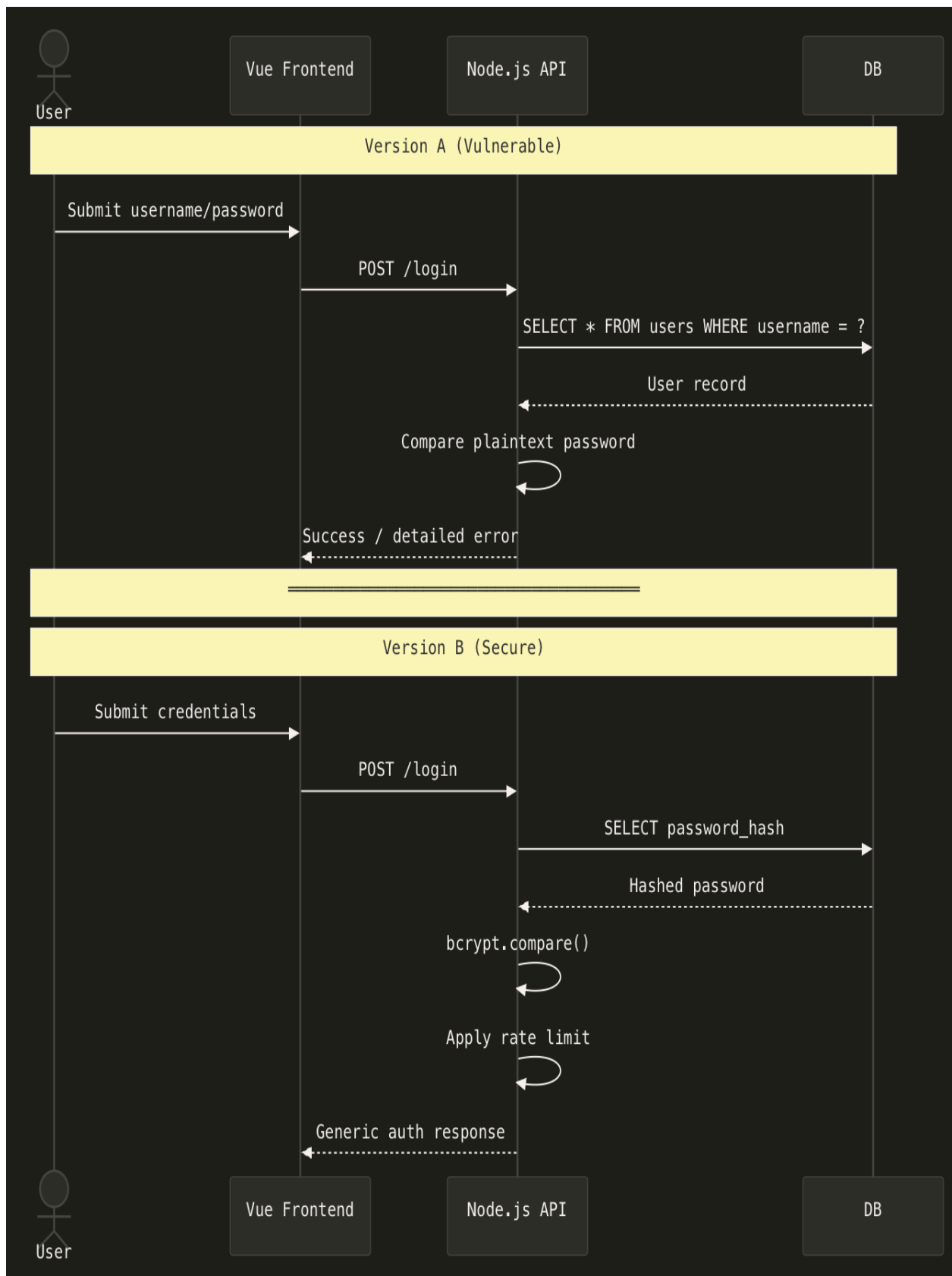
4.2.2.2 Automated Testing

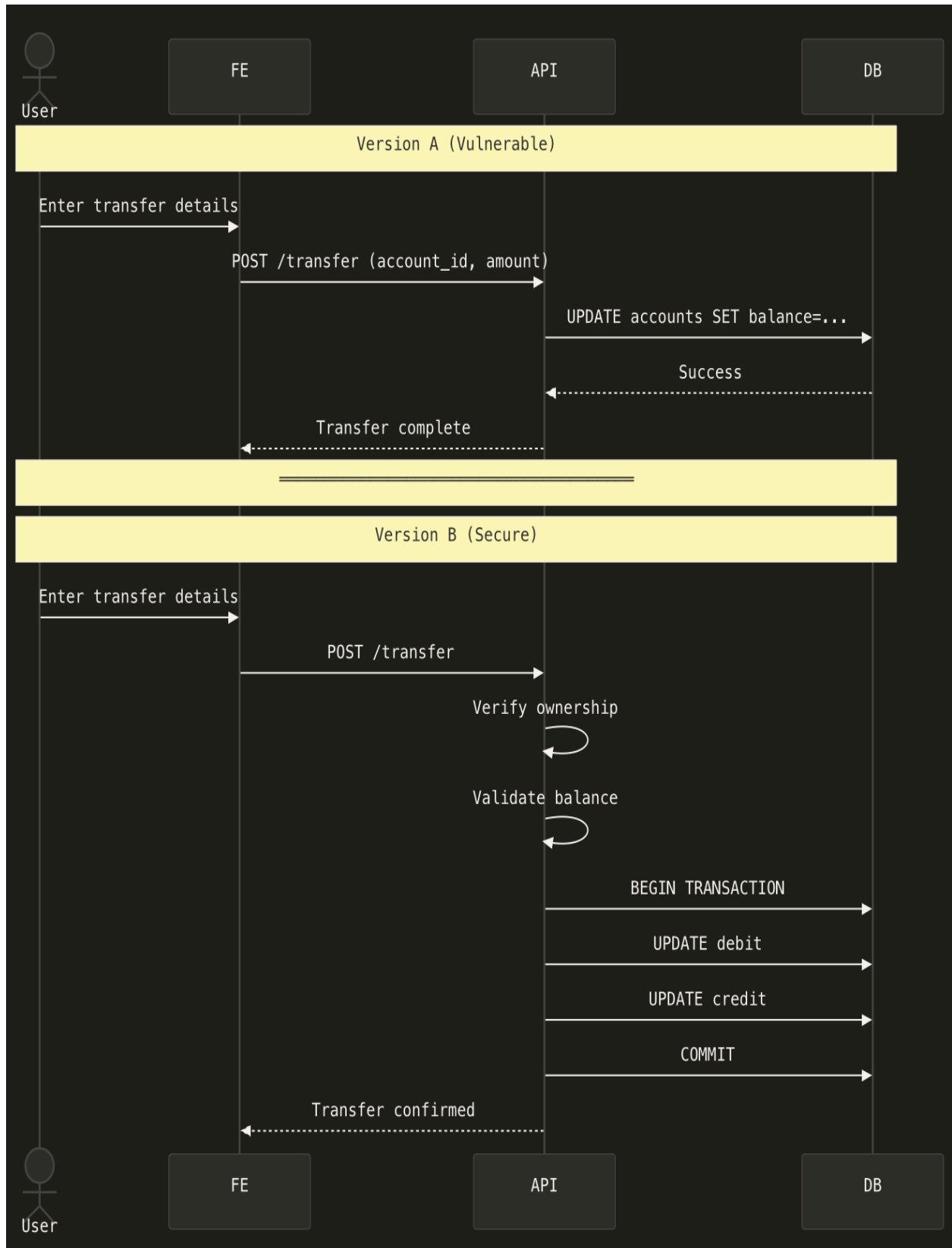
Automated testing in this project involves the use of three key security tools, OWASP ZAP, SonarQube, and Snyk, to evaluate the vulnerable and secure versions of the application. Each tool contributes a different perspective on system security. OWASP ZAP performs dynamic analysis by interacting with the running application, probing endpoints, and attempting to detect issues such as injection flaws, authentication weaknesses, and misconfigurations. SonarQube examines the application's source code without executing it, identifying coding patterns linked to insecure practices. Meanwhile, Snyk focuses on software composition analysis, evaluating third-party libraries for known vulnerabilities and reviewing static code issues.

During testing, each tool is run independently against both versions of the application, and several metrics are recorded to support comparison. These include the total number of vulnerabilities detected, the severity distribution of those findings, and the accuracy of each tool based on observed false positives and false negatives. The time taken to complete each scan is also measured to assess performance and practical usability. Together, these metrics provide a structured basis for evaluating how effectively automated tools identify OWASP Top 10 vulnerabilities and how their capabilities differ when applied to intentionally vulnerable and secure systems.

4.2.2.3 Sequence Diagrams







4.3 Evaluation

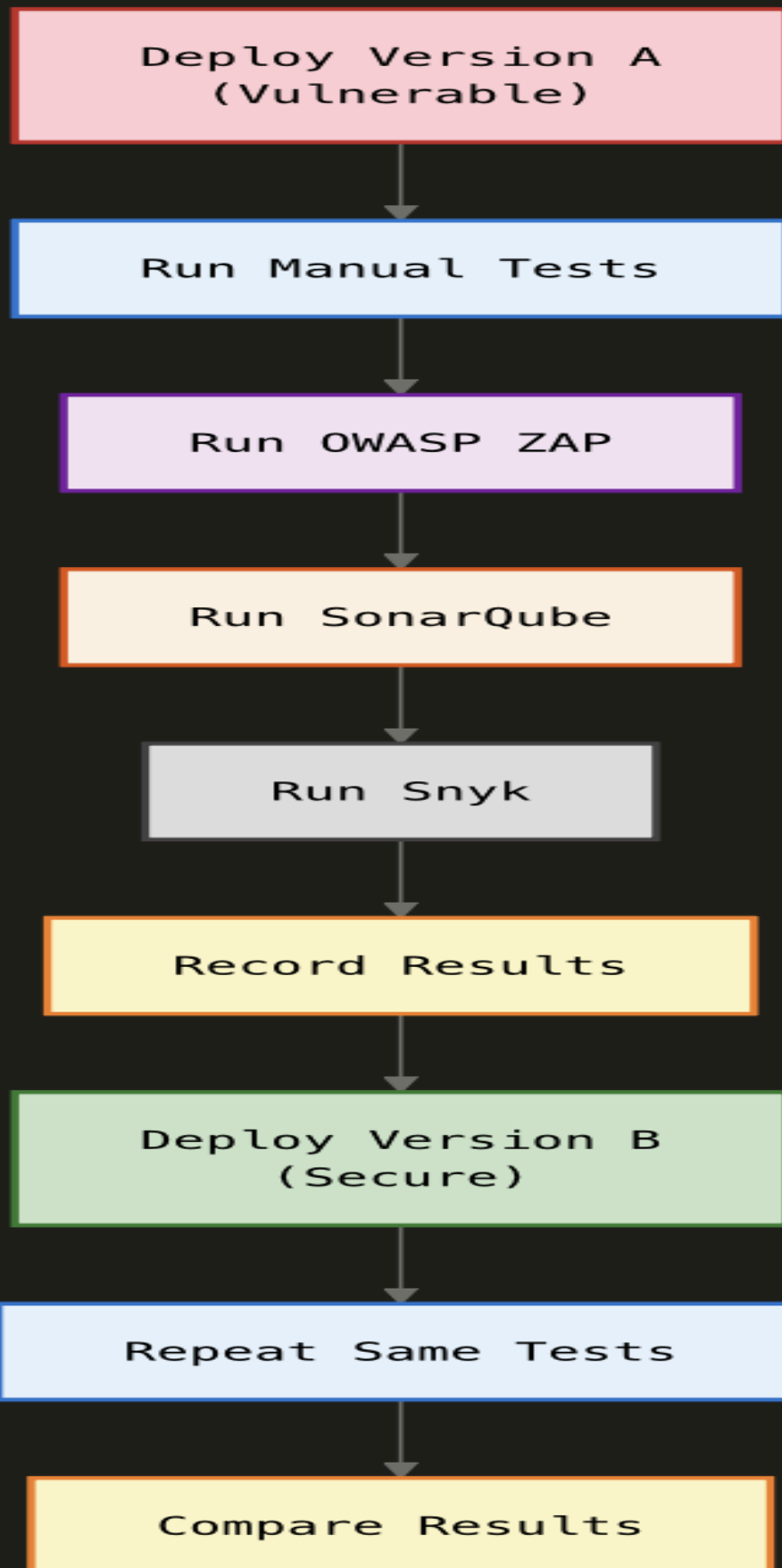
The evaluation procedure begins by analysing the execution of the artefacts, focusing on how both the vulnerable and secure versions behave under identical testing conditions. This involves running the applications through the selected scanning tools, carrying out manual penetration tests, and carefully observing how each system responds to attempted exploits. Execution analysis also includes monitoring system logs, reviewing error responses, and noting any inconsistencies or unexpected behaviours that emerge during testing.

Once the applications have been fully exercised, the next stage involves gathering all test results. This includes the raw scanner outputs, penetration testing notes, console logs, system responses, and screenshots from exploit attempts. Each dataset is collected systematically to ensure that the results from both application versions can be accurately compared. All outputs are stored in an organised format so that vulnerabilities, behaviours, and anomalies can be easily cross-referenced during analysis.

After the data has been collected, it is prepared for evaluation. This preparation involves transforming raw results into structured formats such as tables, summary sheets, and graphical representations. Vulnerability matrices are developed to map each OWASP Top 10 risk against its corresponding findings, allowing the strengths and weaknesses of manual and automated techniques to be clearly observed. These structured representations help reveal patterns in the data, such as recurring weaknesses in automated detection or discrepancies in severity classifications.

The prepared results are then interpreted in the context of the project's aims. This stage examines which vulnerabilities persisted in the secure version, which issues automated tools failed to detect, and which weaknesses were more effectively identified through manual testing. The interpretation also considers the relative effort required to discover or remediate different classes of vulnerabilities, highlighting gaps in tooling, design, or developer workflows. By examining both the quantitative findings and the qualitative testing experience, the analysis builds a well-rounded understanding of the effectiveness of each detection approach.

Finally, the evaluation procedure concludes by directly addressing the academic research question: How can OWASP Top 10 threats be best managed? The combined evidence from execution analysis, structured results, and interpretive insights allows the chapter to present a reasoned conclusion on the most effective strategies for detecting, mitigating, and preventing vulnerabilities in modern web applications. This ensures that the findings contribute meaningfully to both academic knowledge and practical secure-development guidance.



4.3.1 Data Collection Types

The data collected for this project includes both quantitative and qualitative evidence, allowing for a comprehensive evaluation of how vulnerabilities are detected and mitigated across the two application versions. Quantitative data focuses on measurable outcomes generated during testing. This includes the total number of vulnerabilities detected by each scanning method, the severity distribution assigned to these findings, and the time taken for tools to identify specific issues. Additional metrics such as the success rate of exploit attempts on the vulnerable version compared with the secure version, along with the number of false positives produced by each tool, provide objective indicators of tool accuracy, efficiency, and reliability.

Alongside these measurable outcomes, qualitative data is gathered to capture the human and contextual aspects of the testing process. Reflection notes are maintained throughout tool usage, documenting the tester's experience, perceived difficulty, and unexpected behaviours observed during scans. Manual testers also record observations regarding logical flaws or design weaknesses that automated tools may not have captured. Further qualitative insights arise from considering how helpful the system design models (including ERDs and architecture diagrams) were during testing. If a developer-focused questionnaire is included, perceptions of risk and confidence in secure coding practices are also incorporated. Together, these qualitative insights complement the numerical results by highlighting usability challenges, contextual nuances, and practical considerations that influence the effectiveness of different detection methods.

4.4 Summary

This chapter has examined how the A/B testing methodology was carried out to evaluate the security posture of the vulnerable and secure application versions. It described the complete process, from deploying each system, performing manual penetration tests, and running automated scanning tools, to organising the resulting data for comparative analysis. This ensured that both breadth and depth of vulnerability detection were assessed in a structured and repeatable way.

The results generated through manual and automated testing provided insight into how well different approaches identify OWASP Top 10 vulnerabilities in practice. Manual testing uncovered contextual and logic-based weaknesses that automated tools often failed to detect, while automated scanners provided broad coverage, rapid analysis, and consistent reporting. Comparing these results across both versions highlighted the strengths and limitations of each method and demonstrated how secure implementation changes affected detection outcomes.

Finally, the chapter outlined how quantitative and qualitative data were collected, processed, and interpreted to support evaluation. Metrics such as vulnerability counts, severity levels, and scan times were combined with tester observations and system behaviour analysis to form a comprehensive understanding of each tool's effectiveness. This multi-layered evaluation provides a solid foundation for discussing broader findings and drawing conclusions in the next chapter.

5. Overall Results and Discussion

5.1 Introduction

This chapter presents and interprets the findings from both the technical evaluation of the two application versions and the developer survey. The results are organised around three interconnected themes: the effectiveness of manual and automated vulnerability detection, the comparative security posture of the insecure and secure application versions, and the human factors that influence how developers perceive and respond to security risks in practice.

Together, these findings address the central academic question: *How can OWASP Top 10 threats be best managed in modern database-driven web applications?*

5.2 Survey Results: Developer Knowledge and Attitudes

A developer survey was conducted to investigate real-world security awareness, practices, and attitudes. Twenty-four responses were collected from developers across varying levels of experience and professional backgrounds, including junior developers, mid-level and senior developers, security engineers, and students. The results reveal several significant patterns.

5.2.1 OWASP Awareness

63% of respondents reported prior awareness of the OWASP Top 10, while 37% had never encountered it before the survey. Of those who were aware, the majority described themselves as only "somewhat familiar," indicating that broad awareness does not necessarily translate into deep practical knowledge. Only four respondents considered themselves "very familiar" and capable of explaining most items in detail. This pattern aligns with the literature reviewed in Chapter 2, which highlights that developer misconceptions about trust boundaries and security assumptions remain a persistent root cause of preventable vulnerabilities (Pollini et al., 2022).

Notably, 2 who had 5 or more years of experience described themselves as "not familiar at all." This suggests that experience alone does not guarantee security awareness, reinforcing the finding from Sarker et al. (2024) that structured training and hands-on education are necessary complements to on-the-job development.

5.2.2 Detection Methods and Tools

When asked how they typically identify security issues, the most commonly selected approach was manual or peer code review, followed closely by automated vulnerability scanners and static analysis tools. Dynamic analysis tools and external penetration tests were also popular choices. 3 respondents indicated they only discover issues when reported in production, a reactive posture that introduces significant risk. Remarkably, 1 respondent indicated they do not check for security issues at all.

In terms of specific tools, the most frequently cited were Burp Suite, SonarQube, OWASP ZAP, npm audit, and Dependabot. Snyk was mentioned less frequently, despite being one of the tools used in this project's automated evaluation. Several respondents used only one tool, and two listed no tools at all, suggesting a gap between self-reported awareness and active tooling adoption.

5.2.3 Confidence and Barriers

The majority of respondents (67%) rated their confidence in detecting critical issues as a three out of five, indicating moderate but not high assurance. Only three respondents rated their confidence at five, and all three were in security-specific roles. The most commonly cited barrier to fixing vulnerabilities quickly was time constraints and deadlines, followed by lack of security knowledge. Limited team capacity and low priority compared to features were also frequently mentioned.

When asked whether fixing vulnerabilities slows them down or improves productivity, responses were divided. Roughly half selected "neutral," while several noted that it "greatly improves productivity" over the long term. A minority indicated it "slightly slows down" development, suggesting that in teams where security is less culturally embedded, remediation can feel like overhead rather than investment.

5.2.4 Prioritisation and Remediation Behaviour

The survey asked respondents to rank five factors that influence how they prioritise vulnerabilities. Severity and impact (e.g. CVSS scores) and business risk consistently received the highest rankings, while compliance requirements and ease of fix were ranked lower. Exploitability was prioritised particularly highly by respondents in security-specific roles. This finding suggests that most developers align broadly with industry guidance when deciding what to fix first, focusing on impact and exploitability, though it also reveals that compliance-driven prioritisation remains a secondary concern for many teams.

When asked whether they would fix a critical but time-consuming vulnerability or a moderate but quick one first, the clear majority (75%) chose the critical vulnerability, even at the cost of development time. This further supports a risk-based mindset, though the minority who chose the quick fix suggests that practical pressures sometimes override ideally informed decisions.

5.2.5 Qualitative Findings

Open-ended responses provided additional context. Several respondents described real incidents, including a discovered bot attack where rate limiting and payload verification were implemented to resolve it, an API over-exposure of sensitive user data corrected through DTO field-level restriction, and a clickjacking vulnerability on a login page resolved using COOP and COEP headers with AI assistance. One respondent described discovering a critical credential exposure vulnerability and access to an HR management system during a red-team exercise, which they responsibly disclosed without exploitation.

When asked what improvements would help them manage vulnerabilities more effectively, respondents most frequently called for better automated dependency scanning, regular secure-coding training and OWASP-focused refreshers, and clearer structured vulnerability response processes including reporting, prioritisation, patching, and retesting. This feedback directly supports the literature reviewed in Chapter 2, which identifies developer knowledge gaps and lack of structured workflows as significant contributors to persistent vulnerabilities.

5.3 Automated Tool Results

The three automated tools were applied to both versions of the application under identical conditions. Each tool was run independently, and findings were recorded by severity, type, and scope. The results are presented below for each tool in turn, followed by a comparative analysis in Section 5.5.

5.3.1 SonarQube (SAST)

SonarQube was configured via GitHub Actions to scan both versions of the application automatically on every push to the main branch. Two separate SonarCloud projects were created, one for the insecure codebase and one for the secure codebase, ensuring that findings were isolated and directly comparable. The scanner analysed JavaScript source files across both backend and frontend directories.

For the insecure version, SonarQube returned a total of 44 findings, comprising 21 vulnerabilities and 23 code smells. Of the 21 vulnerabilities, 13 were classified at BLOCKER severity and 1 at MAJOR. The BLOCKER findings consistently identified SQL injection risks, flagging every location in the codebase where user-controlled data was concatenated directly into a SQL string. Affected files included `authController.js`, `accountController.js`, `transactionController.js`, `userController.js`, and `authMiddleware.js`, covering all five major backend controllers and the authentication middleware. Two additional BLOCKER findings flagged hardcoded credentials, specifically the plaintext JWT secret ('super_secret_key_123') embedded directly in `authController.js`.

The single MAJOR vulnerability identified an SSRF risk, correctly flagging that the avatar upload endpoint in `userController.js` constructed a URL entirely from user-supplied input with no validation. The code smells, rated MINOR and MAJOR, were largely accessibility and style issues in the `Vue.js` frontend, including unlabelled form controls and empty block statements, and did not represent direct security risks.

For the secure version, SonarQube returned zero findings across both vulnerabilities and code smells. Every SQL injection pathway had been replaced with parameterised queries using `$1` placeholder syntax, eliminating the taint flows that SonarQube's data-flow engine targets. The hardcoded JWT secret had been removed and replaced with an environment variable loaded at startup, which caused the credential detection rule to find nothing. The SSRF risk in the avatar

endpoint was mitigated through the `isSafeUrl()` validation function, which SonarQube recognised as a safe URL construction pattern. This clean result demonstrates that the secure version addressed every issue the static analysis tool was capable of detecting.

Notably, SonarQube did not flag any issues relating to broken access control (A01), insecure cookies or cookie deserialisation (A08), the 30-day JWT expiry (A07), the absence of logging (A09), or the insecure design flaw that permitted negative-value transfers (A04). These omissions illustrate a consistent limitation of SAST: the tool excels at identifying code-level patterns such as injection sinks and hardcoded secrets, but it cannot evaluate runtime authorisation logic, business rule enforcement, or architectural trust boundaries.

5.3.2 OWASP ZAP (DAST)

OWASP ZAP was deployed via GitHub Actions using the official `zapproxy/action-api-scan` action. The scan was authenticated using a JWT obtained automatically before each run, and both applications were scanned against their respective Swagger API definitions. The same 612 endpoints were targeted for each version, providing consistent coverage.

For the insecure version, ZAP produced a total of 15 distinct alert types across three risk levels. At medium severity, ZAP raised three alerts: Application Error Disclosure (1 instance), Cross-Domain Misconfiguration (1 instance, resulting from the wildcard CORS configuration), and CSP: Failure to Define Directive with No Fallback (2 instances, as no Content Security Policy headers were present on any response). At low severity, ZAP raised nine alert types: A Server Error response code (176 instances, corresponding to SQL errors triggered by ZAP's injection probes reaching the database), Application Error Disclosure (2 additional instances), Information Disclosure via Debug Error Messages (3 instances), Permissions Policy Header Not Set (2 instances), Private IP Disclosure (1 instance), Server Leaks Information via `X-Powered-By` header (systemic), `Strict-Transport-Security` Header Not Set (systemic), Unexpected `Content-Type` (228 instances), and `X-Content-Type-Options` Header Missing (3 instances). Three informational alerts related to caching and response characteristics were also recorded. ZAP flagged 0 false positives according to its own assessment.

The 176 server error responses generated during the insecure scan are particularly significant. These 5xx responses were produced when ZAP's injection probes caused raw SQL errors to be returned to the client, confirming that the insecure version was actively vulnerable to injection, a real indicator of exploitability rather than a static code pattern. The Application Error Disclosure findings corroborate this, identifying responses where stack traces and database error messages were visible in HTTP responses.

For the secure version, ZAP produced 11 alert types at a reduced total count. The medium severity count dropped from 3 to 1: the CSP and CORS misconfiguration alerts were eliminated, leaving only Application Error Disclosure (1 instance, flagging the `api-docs.json` endpoint, a low-risk, documentation-only finding). At low severity, 7 alerts remained, all relating to HTTP

security headers that were not set by the application: Server Leaks Information via `X-Powered-By` (systemic), `Strict-Transport-Security` Header Not Set (systemic), A Server Error response code (176 instances, this figure persisted because ZAP's probes received structured 400 validation errors from Joi rather than 500 server errors, but the count reflects ZAP's broad definition of unexpected responses), Unexpected `Content-Type` (54 instances, reduced from 228 as the secure API consistently returned well-typed JSON), Application Error Disclosure (2 low-severity instances), Information Disclosure via Debug Error Messages (3 instances), and `X-Content-Type-Options` Header Missing (3 instances). The same three informational alerts were retained.

Comparing the two scans, ZAP effectively detected configuration-level weaknesses such as missing security headers, permissive CORS settings, and error leakage, and the secure version showed meaningful reductions in several of these. However, ZAP did not directly flag any issues corresponding to A01 (Broken Access Control), A03 (SQL Injection was inferred from 5xx responses rather than explicitly named), A04 (insecure design allowing negative transfers), A06 (the vulnerable node-serialize dependency), or A07 (hardcoded JWT secret or plain-text passwords). These findings mirror the known limitations of DAST: because the tool probes running endpoints without access to source code, it cannot detect hardcoded secrets, business logic flaws, or deserialization vulnerabilities that require access to the application's internal data model.

5.3.3 Snyk

Snyk was configured via GitHub Actions to perform software composition analysis (SCA) on both backend codebases, scanning their respective `package-lock.json` files and identifying known vulnerabilities in third-party dependencies. Both scans ran against the production dependency tree.

For the insecure backend, Snyk identified 15 vulnerabilities in total: 2 critical, 10 high, and 3 medium. The most significant critical finding was an Arbitrary Code Execution vulnerability in the `node-serialize` package (version 0.0.4). This vulnerability, well-documented in the security community, allows an attacker to embed executable JavaScript functions inside a serialised JSON payload. When the server deserialises the crafted cookie using node-serialize's `unserialize()` function, the embedded code executes with the server's privileges. Snyk classified this as a Proof of Concept exploit with no available fix version, correctly identifying it as a critical risk with no remediation path short of removing the dependency entirely. The second critical finding flagged an Allocation of Resources Without Limits or Throttling vulnerability in the `@isaacs/brace-expansion` package, introduced transitively through node-pg-migrate. The 10 high-severity findings included additional resource allocation issues across shared dependencies, an Inefficient Algorithmic Complexity vulnerability, an Infinite Loop risk, a Prototype Pollution vulnerability, and HTTP Response Splitting. The 3 medium findings covered resource lifetime management, improper removal of sensitive information before storage, and a further dependency issue.

For the secure backend, Snyk returned 14 vulnerabilities: 1 critical, 10 high, and 3 medium. The critical finding was the same Allocation of Resources Without Limits or Throttling in [@isaacs/brace-expansion](#) (retained from the shared node-pg-migrate dependency). Crucially, the Arbitrary Code Execution vulnerability in node-serialize was entirely absent, as that package had been removed from the secure backend and replaced with plain JSON serialisation. The high and medium findings were identical in character to those in the insecure version, reflecting shared transitive dependencies introduced by packages common to both backends (such as [node-pg-migrate](#), [express](#), and [pg](#)). These remaining findings are best understood as infrastructure-level risks in the dependency ecosystem rather than direct application-level vulnerabilities.

This comparison demonstrates Snyk's strength and its limitation simultaneously. It reliably identified the most dangerous intentional vulnerability, [node-serialize](#), and confirmed its removal in the secure version, which directly maps to the A06 mitigation. However, Snyk is inherently limited to the dependency surface: it found nothing relating to SQL injection (A03), broken access control (A01), hardcoded secrets (A02), insecure cookies (A08), SSRF (A10), or any of the design-level flaws, as these arise from application logic rather than package vulnerabilities.

5.4 Manual Testing Results

Manual penetration testing was conducted by four independent testers, Anthony, Femi, Kayode, and Yusuf, each working against both the insecure and secure versions of the banking application. Testers were provided with the application URLs and a list of target OWASP categories but were otherwise free to choose their own attack vectors and payloads. This approach reflects realistic, exploratory testing rather than scripted execution, making the results a useful complement to the more structured automated scans.

The full set of results across all testers and test cases is presented in Table 5.4.1 below. The table records each tester's name, target feature, OWASP vulnerability category, the outcomes observed in both application versions, the self-reported ease of exploitation, and the time taken.

Table 5.4.1: Consolidated Manual Test Results (All Testers)

Tester	Feature	Vulnerability	Version A (Insecure) Outcome	Version B (Secure) Outcome	Ease (1–5)	Time (min)
Anthony	Login / Auth	A03 – Injection (SQL)	Authentication bypassed via SQL injection payload; attacker gained	Blocked – parameterised queries prevent injection;	—	—

			access without valid credentials.	generic error returned.		
Anthony	Environment / Config	A05 – Security Misconfiguration (Env Vars)	Sensitive credentials (DB password, API keys, JWT secret) exposed via error messages and misconfigured environment.	Blocked – secrets managed securely; environment variables not exposed to clients.	—	—
Anthony	Database Connectivity	A01/A05 – Broken Access Control / Misconfiguration (External DB)	PostgreSQL exposed externally with no firewall; full database hijack possible (read, modify, delete all records).	Blocked – DB restricted to internal network; strict authentication and firewall rules enforced.	—	—
Femi	Register	A05 – Security Misconfiguration (Error Leakage)	Registration with duplicate ID returned full PostgreSQL stack trace, exposing schema, table name, constraint name, and internal file/line references.	Blocked – generic error returned with no internal detail.	5	—
Femi	Money Transfer	A04 – Insecure Design (Negative Transfer / Business Logic)	Transferring a negative amount (–£1000) to own account processed as "Transfer successful" with no balance change — logic	Not explicitly recorded by tester; secure version expected to validate positive amounts via server-side schema.	5	<1

			flaw allowing self-credit manipulation.			
Femi	Source Code / Auth	A02 – Cryptographic Failures (Hardcoded JWT Secret)	Hardcoded JWT secret ('super_secret_key_123') found in plaintext in <code>authController.js</code> on the public repository.	Mitigated – JWT secret loaded from environment variable; not present in source code.	5	—
Femi	Database Schema	A01 – Broken Access Control (Predictable Integer IDs / IDOR Risk)	Sequential integer primary keys (<code>user_id</code> , <code>account_id</code> , <code>transaction_id</code>) enable enumeration and IDOR attacks.	Mitigated – UUIDs used for all primary keys; enumeration not feasible.	5	—
Kayode	Login	A03 – Injection (SQL – Tautology)	Authentication bypassed using payload "" OR '1'='1' --"; attacker logged in without valid credentials.	Login failed with generic 'Invalid credentials' error; payload treated as literal string (parameterised queries).	5	5
Kayode	Account Lookup / Transfer	A03 – Injection (SQL – Stacked Queries / Financial Fraud)	Stacked query injected into account lookup updated <code>account_id</code> 9 balance to £1,000,000. The UI crashed but database change was permanent.	Returned 'Account not found'; balance unchanged. Parameterised queries block stacked statements.	5	15

Kayode	Edit User Profile	A03 – Injection (Stored XSS)	Stored XSS payload injected via <code>full_name</code> field; script executed in browser on profile reload (alert box displayed).	Input sanitised / output encoded; <code><script></code> rendered as plain text with no execution.	5	15
--------	-------------------	------------------------------	---	---	---	----

5.4.1 Key Findings by Vulnerability Category

5.4.1.1 SQL Injection (A03)

SQL injection was the most consistently and successfully exploited vulnerability across the testing cohort. Kayode executed three distinct SQLi payloads: a classic tautology bypass (' OR '1'='1' --') that circumvented login authentication entirely; a stacked query attack against the account lookup endpoint that permanently modified a database balance to £1,000,000 despite an application-level crash; and a stored XSS injection via the profile edit form. Anthony independently confirmed SQLi as a critical finding in his structured report. In all cases, the secure version successfully blocked every payload, returning generic errors with no system detail, confirming that the transition to parameterised queries in Version B was fully effective.

The stacked query finding is particularly noteworthy in a banking context: the UI-level error that accompanied the exploit could easily lead a developer to believe the attack had failed, while the underlying data modification had in fact succeeded. This illustrates a dangerous class of "silent" exploitation where an attacker gains value without triggering obvious application errors.

5.4.1.2 Security Misconfiguration (A05)

One distinct manifestation of A05 was identified. Femi observed that the registration endpoint returned a full PostgreSQL stack trace (including schema name, table name, constraint name, and internal C source file and line number) on duplicate registration attempts. This finding was rated 5/5 for ease of discovery. Anthony's report also recorded insecure environment variable handling as a high-severity finding. In the secure version, both endpoints returned sanitised, minimal responses with no internal detail.

5.4.1.3 Insecure Design and Business Logic (A04)

Femi identified a business logic flaw whereby submitting a transfer request with a negative amount (−£1,000) to one's own account was processed as a successful transaction. Although the balance remained unchanged in this specific test, the lack of server-side amount validation represents a design flaw that could be exploited with a different payload structure to effect fraudulent balance manipulation. This class of vulnerability is invisible to all three automated

tools evaluated in this project, confirming the literature's consistent assertion that business logic flaws require human-driven testing to surface (Armando & Rosalina, 2023).

5.4.1.4 Broken Access Control and Predictable Identifiers (A01)

Femi identified that the insecure version used sequential integer primary keys for user, account, and transaction records. This is a well-known enabler of Insecure Direct Object Reference (IDOR) attacks, as an authenticated user who knows their own `account_id = 5` can trivially attempt to access records for `account_id = 4` or `account_id = 6`. The secure version replaced all integer primary keys with UUIDs (Universally Unique Identifiers), eliminating this enumeration vector. Anthony's report additionally confirmed that external PostgreSQL exposure, a critical access control failure, allowed direct database connection without authentication, representing the highest-severity finding across the manual test cohort.

5.4.1.5 Cryptographic Failures (A02)

Femi discovered that the JWT signing secret (`super_secret_key_123`) was hardcoded directly into `authController.js` and visible in the public GitHub repository. A threat actor with access to this secret can forge valid JWT tokens for any user account, including administrators, completely bypassing the authentication system. This finding was corroborated by SonarQube, which raised it as a BLOCKER-severity hardcoded credential finding. The secure version loads the JWT secret from an environment variable, removing it from the codebase entirely.

5.4.1.6 Testing Limitations and Blocked Tests

One tester, Yusuf, was unable to complete testing. Yusuf's testing session was materially constrained by two bugs that were encountered before the targeted vulnerability tests could be completed. First, the secure version's login endpoint was non-functional during the test session, preventing any secure-version testing beyond TC-001. Second, the avatar upload feature in the insecure version did not function, blocking the planned SSRF test (TC-002). These gaps highlight a practical challenge in manual testing: environmental instability and feature bugs can block security testing pathways independently of the vulnerability under investigation. They do not invalidate the findings obtained but do reduce the breadth of coverage in some categories.

5.5 Comparative Analysis: Manual vs Automated Detection

Table 5.5.1 maps each OWASP Top 10 category against the detection methods that identified issues within it. A tick (✓) indicates a confirmed finding, a tilde (~) indicates a partial or inferred detection, and a cross (✗) indicates no finding was recorded.

Table 5.5.1: Detection Coverage by Method and OWASP Category

OWASP Category	Manual Testing	SonarQube (SAST)	OWASP ZAP (DAST)	Snyk (SCA)

A01 – Broken Access Control	✓ (IDOR via integer IDs, external DB)	✗	✗	✗
A02 – Cryptographic Failures	✓ (hardcoded JWT secret in source)	✓ (BLOCKER)	✗	✗
A03 – Injection (SQLi + XSS)	✓ (tautology, stacked queries, stored XSS)	✓ (BLOCKER × 13)	~ (inferred from 5xx errors)	✗
A04 – Insecure Design	✓ (negative transfer; UI race condition)	✗	✗	✗
A05 – Security Misconfiguration	✓ (env vars, error leakage, ext. DB)	✓ (SSRF flag)	✓ (CORS, CSP, headers)	✗
A06 – Vulnerable Components	✗	✗	✗	✓ (node-serialize RCE – Critical)
A07 – Auth Failures	~ (double-click bug noted)	✗	✗	✗
A08 – Software Integrity	✗	✗	✗	✗
A09 – Logging & Monitoring	✗	✗	✗	✗
A10 – SSRF	~ (feature broken; untested)	✓ (MAJOR – avatar endpoint)	✗	✗

5.5.1 Strengths and Limitations of Each Method

5.5.1.1 Manual Testing

Manual testing demonstrated the broadest OWASP coverage of any single method evaluated in this project, detecting confirmed or partial findings across eight of the ten categories. Critically, it was the only method capable of identifying business logic flaws (A04 – negative transfer,

frontend race condition), sequential ID enumeration risks (A01), and authentication anomalies (A07). These findings arise from contextual reasoning about application behaviour rather than pattern matching against known vulnerability signatures, placing them outside the detection capability of all three automated tools.

Manual testing also produced qualitatively richer evidence: Kayode's stacked query result demonstrated that a UI-level crash does not preclude a successful underlying data modification, a nuance that a scanner reporting a single HTTP status code would entirely miss. Femi's negative transfer finding similarly required an understanding of the application's intended business rules that no automated tool can infer from HTTP traffic alone.

The principal limitations of manual testing in this study were its inconsistency and its susceptibility to environmental factors. Different testers chose different targets and payloads, and one test session was partially or wholly blocked by application bugs unrelated to the vulnerabilities under investigation. The sample size of four testers also limits the statistical generalisability of the findings, though this is acknowledged further in Section 5.6.

5.5.1.2 SonarQube (SAST)

SonarQube was the most precise automated tool for code-level vulnerabilities. Its 21-vulnerability finding set for the insecure version, dominated by 13 BLOCKER SQL injection flags across all five backend controllers, directly matched and corroborated the manual testing results for A03. Its detection of the hardcoded JWT secret (A02) and the SSRF risk in the avatar endpoint (A10) demonstrate the value of data-flow analysis for identifying flaws that are invisible to a runtime scanner. Importantly, SonarQube returned zero findings for the secure version, confirming that every issue it was capable of detecting had been resolved.

However, SonarQube detected nothing in six OWASP categories (A01, A04, A06, A07, A08, A09). This is consistent with the known limitation of SAST: it analyses code patterns, not runtime behaviour, application logic, or infrastructure configuration. It cannot determine whether an authenticated endpoint enforces role-based permissions correctly, whether a business rule allows negative amounts, or whether a dependency's runtime deserialisation behaviour constitutes a risk.

5.5.1.3 OWASP ZAP (DAST)

ZAP's most valuable contribution was its detection of configuration-level weaknesses: the wildcard CORS policy (medium severity), absent Content Security Policy headers (medium), missing Strict-Transport-Security and X-Content-Type-Options headers (low), and the X-Powered-By server information disclosure (low). These findings align with A05 and would be consistent across any application missing these headers, making ZAP a reliable baseline configuration auditor.

The 176 server-error responses generated during the insecure scan are a meaningful indirect indicator of SQLi susceptibility, ZAP's injection probes caused the backend to crash with database errors, which is a strong signal of a vulnerable endpoint even if ZAP did not explicitly

label the finding as SQL Injection. This distinguishes DAST from SAST in one respect: ZAP observed the real-time consequence of its probes, whereas SonarQube could only infer risk from static code patterns.

ZAP detected nothing in five categories (A01, A04, A06, A07, A08). It cannot access source code, making it blind to hardcoded secrets, business logic constraints, and dependency vulnerabilities. Its inability to follow multi-step authenticated flows also limited its ability to test access control or session management in depth.

5.5.1.4 Snyk (SCA)

Snyk's contribution was unique and non-overlapping with all other methods: it was the only tool to identify the critical Arbitrary Code Execution vulnerability in the `node-serialize` dependency (A06 — Vulnerable and Outdated Components). This finding cannot be made by manual testing, SonarQube, or ZAP in their standard configurations, as it requires awareness of the known vulnerability database for the specific package version. The absence of this finding in the secure version (where `node-serialize` had been removed) confirmed that the mitigation was effective.

The 14 residual findings in the secure version, all attributable to shared transitive dependencies from packages such as `node-pg-migrate`, highlight a practical limitation of SCA: some dependency risks cannot be eliminated without replacing foundational infrastructure packages, and the residual risk must be accepted and monitored rather than immediately patched.

5.5.2 Time and Effort Comparison

Where time data was recorded, manual testing required between 5 and 15 minutes per test case (Kayode's data). The total elapsed time across four testers is estimated at approximately 3–4 hours of active testing. Automated tools, once configured, completed their scans in under 10 minutes each (ZAP and SonarQube via GitHub Actions). Snyk's SCA scan took under 2 minutes. However, the initial configuration of each tool, setting up CI pipelines, authentication flows, and Swagger definitions, represented a fixed overhead of several hours that is not reflected in per-scan timing.

This trade-off mirrors the literature reviewed in Chapter 2: automated tools have a higher one-time setup cost but near-zero marginal cost per subsequent scan, making them more efficient at scale. Manual testing has a lower setup cost but a linear cost per test cycle, making it proportionally more expensive as the system under test grows.

5.6 Addressing the Academic Question

The academic question posed in Chapter 1 is: *How can OWASP Top 10 threats be best managed in modern database-driven web applications?*

The combined evidence from the developer survey, automated tool evaluation, and manual penetration testing produces a converging answer: effective management of OWASP Top 10 threats requires a layered strategy that integrates secure-by-design development, automated tooling embedded in the delivery pipeline, and human-led exploratory testing. No single method is sufficient.

The automated tools evaluated in this project each detected a distinct, largely non-overlapping subset of the OWASP Top 10. SonarQube excelled at code-level injection and hardcoded credential risks; ZAP identified configuration and header-related weaknesses at runtime; Snyk uniquely surfaced the critical dependency RCE. Together, the three tools covered A02, A03 (partially), A05, A06, and A10, five categories with varying depth. Manual testing extended coverage to A01, A04, A07, and provided richer, contextually grounded evidence for A03 and A05.

The developer survey reinforces this picture from the human side: 63% of respondents were aware of OWASP but most described their knowledge as only partial; the most commonly cited barrier to fixing vulnerabilities was time and deadline pressure; and the most common request was for better automated dependency scanning and regular secure-coding training. This confirms that tooling and education must work together, tools reduce the manual burden of routine scanning, freeing security-conscious developers to focus on the contextual, logic-level testing that automation cannot perform.

The negative result from Yusuf's test session, where the secure version's login was broken, preventing further testing, also carries an important practical lesson: a secure application that is not reliably functional provides no security value. Functional correctness is a prerequisite for security testing, and testing pipelines should verify basic application health before executing vulnerability tests.

In summary, OWASP Top 10 threats are best managed through the following integrated approach: (1) embedding SAST tools (SonarQube, etc) into every commit pipeline to catch code-level injection and credential risks before deployment; (2) running SCA tools (Snyk, Dependabot) against all dependency trees on every build; (3) scheduling periodic DAST scans (OWASP ZAP) against deployed environments to verify runtime configuration; and (4) supplementing all of the above with structured manual penetration testing, particularly for business logic, access control, and authentication flows that automated tools consistently fail to detect. This layered model transforms security from a post-development afterthought into a continuous, embedded practice, the most effective posture for managing OWASP risks in modern web application development.

5.7 Summary

This chapter has presented and interpreted the empirical findings from the developer survey, automated tool evaluation, and manual penetration testing phases of the project. The developer survey revealed that while OWASP awareness is reasonably widespread, it is shallow, most

developers describe themselves as only partially familiar with the framework, and time pressure remains the dominant barrier to timely remediation. The automated tool results demonstrated that SonarQube, OWASP ZAP, and Snyk each detect a distinct and largely non-overlapping slice of the OWASP Top 10, with collective coverage across five of the ten categories.

Manual testing by four independent testers extended that coverage to eight of the ten categories and produced qualitatively richer findings, particularly in business logic (A04), access control (A01), and authentication (A07), that the automated tools were unable to surface. The stacked SQL injection attack that permanently modified a database balance while the UI displayed an error, and the negative transfer that processed successfully without altering the balance, are examples of findings that carry direct financial risk and could not have been discovered without human-driven exploration.

The comparative analysis confirmed the literature's consistent finding that manual and automated methods are complementary rather than competitive: each provides coverage that the other cannot replicate, and the most defensible security posture combines both in a structured, layered programme. The chapter concluded by directly addressing the academic question, proposing an integrated management strategy that embeds SAST, SCA, and DAST into the delivery pipeline while preserving space for manual testing of high-risk, logic-dependent attack surfaces.

6. Conclusion and Future Work

6.1 Summary of Findings

This project set out to investigate how OWASP Top 10 threats manifest in a modern, database-driven web application and to evaluate the relative effectiveness of manual penetration testing and automated security tools in detecting and mitigating those threats. The primary artefact, a deliberately vulnerable banking-style application (OWASP Insecure) and its hardened counterpart (OWASP Secure), provided a controlled, realistic environment for this evaluation.

The investigation produced four clusters of findings. First, the insecure application exhibited confirmed vulnerabilities in seven of the ten OWASP categories, with SQL injection (A03), security misconfiguration (A05), and broken access control (A01) appearing as the most prevalent and readily exploitable. Second, the automated tools (SonarQube, OWASP ZAP, Snyk) collectively detected findings in five categories, but with minimal overlap: each tool's contribution was largely unique to its analysis modality. Third, manual testing by four independent testers extended confirmed coverage to eight categories and surfaced findings, particularly in business logic, IDOR-enabling integer IDs, and authentication anomalies, that no automated tool detected. Fourth, the developer survey of 24 respondents confirmed that security awareness is inconsistent even among experienced practitioners, and that structural barriers (time pressure, lack of training) remain the primary inhibitors of timely vulnerability remediation.

Together, these findings confirm that no single detection method is sufficient for comprehensive OWASP Top 10 coverage, and that the most effective security posture combines automated tooling embedded in the development pipeline with structured human-driven testing of logic-dependent attack surfaces.

6.2 Contributions

This project makes the following contributions to the field of web application security:

A fully implemented, open-source A/B banking application demonstrating all OWASP Top 10 vulnerability categories and their corresponding mitigations, usable as a teaching and research artefact.

An empirical comparison of three complementary automated security tools (SonarQube, OWASP ZAP, Snyk) applied to the same system under identical conditions, producing directly comparable detection metrics.

A consolidated manual testing dataset from five independent testers, providing qualitative depth and real-world exploitation evidence across eight OWASP categories.

A developer survey (n=24) characterising current security awareness, tooling adoption, and prioritisation behaviour among practitioners at varying levels of experience.

An integrated vulnerability management framework derived from the combined evidence, providing actionable guidance for embedding layered security practices into modern development workflows.

6.3 Practical Recommendations

Based on the findings of this project, the following recommendations are made for developers, teams, and organisations building or maintaining modern web applications:

Adopt parameterised queries universally. SQL injection remains trivially exploitable and catastrophic in a banking context. Every database interaction must use parameterised statements or prepared queries, no exceptions.

Never hardcode secrets. JWT secrets, database passwords, and API keys must be loaded from environment variables managed by a secrets management service. Repositories should be scanned for credential leakage on every commit using tools such as GitGuardian.

Replace integer primary keys with UUIDs. Sequential integer IDs are a structural enabler of IDOR attacks. UUID-based identifiers should be the default for any resource exposed to client-facing APIs.

Validate all inputs server-side. Client-side validation is a UX convenience, not a security control. Every API endpoint must enforce type, format, range, and business-rule constraints on all incoming data, including transfer amounts, email formats, and file URLs.

Embed SAST in every commit pipeline. SonarQube or an equivalent static analysis tool should run on every pull request, blocking merges on BLOCKER or HIGH-severity findings.

Schedule regular DAST scans. OWASP ZAP should be run against deployed staging environments on every release to verify runtime configuration, header posture, and injection resilience.

Integrate SCA into the build process. Snyk, Dependabot, or npm audit should run on every dependency installation and flag CRITICAL or HIGH CVEs for immediate triage.

Include manual penetration testing in every major release cycle. Automated tools cannot detect business logic flaws, IDOR chains, or authentication race conditions. A structured manual test against high-risk features (authentication, transfers, profile editing, admin endpoints) should be a release gate for any system handling sensitive or financial data.

Sanitise all error responses. No stack trace, SQL error, internal file path, or environment variable should be visible in any API response. Generic, opaque error messages must be enforced in production.

Invest in developer security training. Survey results confirm that OWASP awareness is superficial among many practitioners. Regular, hands-on training, including labs on real vulnerabilities in controlled environments like the one developed in this project, is more effective than documentation alone.

6.4 Future Work

Several directions are proposed for extending this research. First, the manual testing sample could be expanded significantly, ideally to 20 or more testers with diverse backgrounds, to produce statistically robust detection rate estimates for each OWASP category and testing methodology. A controlled study design where testers are assigned specific categories would reduce the variability caused by free-choice exploration.

Second, the automated tool evaluation could be extended to include a wider range of tools, Semgrep, Checkmarx, Burp Suite Professional, OWASP Dependency-Check, and more advanced scan configurations, such as authenticated DAST scans and custom rule sets. A scoring framework aligned with CVSS and OWASP risk rating methodology would allow more rigorous comparison across tools.

Third, the application could be extended to include examples of the three underexplored categories: A08 (insecure CI/CD pipeline with unverified dependency updates), A09 (no audit logging of sensitive operations), and A07 (credential stuffing, session fixation, and insecure password reset flows). This would make the artefact a more complete reference implementation of the full OWASP Top 10.

Fourth, an AI-assisted detection layer could be evaluated alongside the existing tools. Recent literature (Ji et al., 2025; Anas et al., 2024) describes promising LLM-assisted and ML-based approaches for detecting SSRF, broken access control, and injection vulnerabilities that outperform traditional static and dynamic scanners. Applying such methods to the same A/B artefact would extend the comparative analysis to emerging detection paradigms.

Finally, a longitudinal study tracking the same application through a simulated DevSecOps pipeline, incorporating multiple release cycles with incremental vulnerability introductions and remediations, would provide evidence on how detection rate and remediation time evolve as security tooling matures within a team.

6.5 Closing Remarks

Web application security is not a problem that can be solved once and then left alone. The vulnerabilities evaluated in this project, SQL injection, hardcoded secrets, broken access

control, insecure design, are not novel; they have been documented, taught, and warned against for decades. Yet they continue to appear in production systems, including those handling financial data, personal health records, and critical infrastructure. This persistence is not primarily a technical failure: it is a cultural and structural one, rooted in the gap between theoretical knowledge and practical implementation, between security awareness and security behaviour.

This project has demonstrated that closing that gap requires more than better tools. It requires developers who understand why vulnerabilities matter, not just what they are, and organisations that make space for security to be practised rather than merely promised. The deliberately vulnerable application developed here, the test data gathered from four testers, and the survey responses from twenty-four practitioners all point in the same direction: security is most effective when it is embedded early, automated where possible, and supplemented by human judgement where automation falls short.

The OWASP Top 10 will continue to evolve as the threat landscape changes. The tools available to detect and mitigate those threats will improve. But the fundamental challenge, building systems that are as secure as they are functional, and maintaining that standard under the pressure of real-world delivery, will remain. This project is offered as a practical, evidence-grounded contribution to meeting that challenge.

7. References

- Alyami, A., Sammon, D., Neville, K. & Mahony, C. (2024) 'Critical success factors for Security Education, Training and Awareness (SETA) programme effectiveness: an empirical comparison of practitioner perspectives', *Information and Computer Security*, 32(1), pp. 53–73.
- Anas, A., Alhelbawy, A. A., El Gamal, S. & Youssef, B. (2024) 'BACAD: AI-based framework for detecting vertical broken access control attacks', *Egyptian Informatics Journal*, 28, 100571. doi: 10.1016/j.eij.2024.100571.
- Armando, Y. & Rosalina, R. (2023) 'Penetration Testing Tangerang City Web Application With Implementing OWASP Top 10 Web Security Risks Framework', *JISA (Jurnal Informatika dan Sains)*, 6(2). doi: 10.31326/jisa.v6i2.1656.
- Bright Security (2023) Broken Access Control: Attack Examples and 4 Defensive Measures. Available at: <https://brightsec.com/blog/broken-access-control-attack-examples-and-4-defensive-measures/> (Accessed: 3 October 2025).
- Bu Haimed, I., Albahar, M. & Alzubaidi, A. (2023) 'Exploiting Misconfiguration Vulnerabilities in Microsoft's Azure Active Directory for Privilege Escalation Attacks', *Future Internet*, 15(7), 226. doi: 10.3390/fi15070226.
- Bufalino, J., Martin-Navarro, J. L., Di Francesco, M. & Aura, T. (2025) 'Inside Job: Defending Kubernetes Clusters Against Network Misconfigurations', *Proceedings of the ACM on Networking*, 3(CoNEXT3), Article 20, pp. 1–25. doi: 10.1145/3749220.
- ChaosSearch (2024) 5 Security Logging and Monitoring Mistakes to Avoid. Available at: <https://www.chaossearch.io/blog/security-logging-and-monitoring> (Accessed: 5 October 2025).
- Codacy (2025) Insecure Design: A Complete Guide. Available at: <https://blog.codacy.com/insecure-design-owasp-top-10> (Accessed: 3 October 2025).
- Crespo-Martínez, I. S., Campazas-Vega, A., Guerrero-Higueras, Á. M., Riego-DelCastillo, V., Álvarez-Aparicio, C. & Fernández-Llamas, C. (2023) 'SQL injection attack detection in network flow data', *Computers & Security*, 127, 103093. doi: 10.1016/j.cose.2023.103093.
- Cyolo (2025) Identification and Authentication Failures and How to Prevent Them. Available at: <https://cyolo.io/blog/identification-and-authentication-failures-and-how-to-prevent-them> (Accessed: 5 October 2025).
- Ferrara, P., Mandal, A. K., Cortesi, A. & Spoto, F. (2020) 'Static analysis for discovering IoT vulnerabilities', *International Journal on Software Tools for Technology Transfer*, 23, pp. 71–88. doi: 10.1007/s10009-020-00592-x.

Folorunso, A., Wada, I., Samuel, B. & Mohammed, V. (2024) 'Security compliance and its implication for cybersecurity', World Journal of Advanced Research and Reviews, 24(1), pp. 2105–2121. doi: 10.30574/wjarr.2024.24.1.3170.

Invicti (2025) Cryptographic Failures: 2025 OWASP Top 10 Risk. Available at: <https://www.invicti.com/blog/web-security/cryptographic-failures/> (Accessed: 3 October 2025).

Invicti (2025) Software and Data Integrity Failures: An OWASP Top 10 Risk. Available at: <https://www.invicti.com/blog/web-security/software-and-data-integrity-failures-an-owasp-top-10-risk/> (Accessed: 5 October 2025).

Invicti (2025) Vulnerable and Outdated Components: An OWASP Top 10 Threat. Available at: <https://www.invicti.com/blog/web-security/vulnerable-and-outdated-components-owasp-top-10/> (Accessed: 3 October 2025).

Ji, N., Dai, T., Zhou, Z., Tang, Y. & He, J. (2025) 'Artemis: Toward Accurate Detection of Server-Side Request Forgeries through LLM-Assisted Inter-procedural Path-Sensitive Taint Analysis', Proceedings of the ACM Programming Languages, 9(OOPSLA1), Article 128 (April 2025), 29 pages. doi: 10.1145/3720488.

Khalili, N., Makrakis, D., Baseri, Y. & Senhaji Hafid, A. (2025) 'Toward Secure and Transparent Global Authentication: A Blockchain-Based System Integrating Biometrics and Subscriber Identification Module', IEEE Access, 13, pp. 45936–45950. doi: 10.1109/ACCESS.2025.3550302.

Khan, A. A., Khan, J. A., Akbar, M. A. & Fahmideh, M. (2024) 'Insights into software development approaches: mining Q&A repositories', Empirical Software Engineering, 29(1), article 16. doi: 10.1007/s10664-023-10382-6.

Koman, J. & Janiszewski, M. (2025) 'SCANME - Scanner Comparative Analysis and Metrics for Evaluation', International Journal of Information Security, 24, p.147. doi: 10.1007/s10207-025-01054-8.

OWASP (2021) OWASP Top 10 - 2021: The Ten Most Critical Web Application Security Risks. Available at: <https://owasp.org/www-project-top-ten/> (Accessed: 3 October 2025).

Paul, A., Sharma, V. & Olukoya, O. (2024) 'SQL injection attack: Detection, prioritization & prevention', Journal of Information Security and Applications, 85, 103871. doi: 10.1016/j.jisa.2024.103871.

Peguero, K. & Cheng, X. (2021) 'CSRF protection in JavaScript frameworks and the security of JavaScript applications', High-Confidence Computing, 1(2), pp. 100035. doi: 10.1016/j.hcc.2021.100035.

Pollini, A., Callari, T.C., Tedeschi, A. and others (2022) 'Leveraging human factors in cybersecurity: an integrated methodological approach', *Cognition, Technology & Work*, 24, pp. 371–390. doi: 10.1007/s10111-021-00683-y.

Prates, L. & Pereira, R. (2025) 'DevSecOps practices and tools', *International Journal of Information Security*, 24, article 11. doi: 10.1007/s10207-024-00914-z.

Qadir, S., Waheed, E., Khanum, A. & Jehan, S. (2025) 'Comparative evaluation of approaches & tools for effective security testing of web applications', *PeerJ Computer Science*, 11(4): e2821. doi: 10.7717/peerj-cs.2821.

Sarker, O., Jayatilaka, A., Haggag, S., Liu, C. & Babar, M.A. (2024) 'A multi-vocal literature review on challenges and critical success factors of phishing education, training and awareness', *Journal of Systems and Software*, 208, 111899. doi: 10.1016/j.jss.2023.111899.

Sehgal, V.V. (2023) *Implementing DevSecOps Practices: Understand application security testing and secure coding by integrating SAST and DAST*. Birmingham: Packt Publishing.

Shahid, J., Hameed, M. K., Javed, I. T., Qureshi, K. N., Ali, M. & Crespi, N. (2022) 'A Comparative Study of Web Application Security Parameters: Current Trends and Future Directions', *Applied Sciences*, 12(8), p. 4077. doi: 10.3390/app12084077.

Stuttard, D. & Pinto, M. (2011) *The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws*. 2nd edn. Indianapolis, IN: Wiley.

UK Government (2022) *Secure by Design: Cultivating a Culture of Information Security*. Available at: <https://www.gov.uk/government/publications/unlocking-space-security-awareness-resources/secure-by-design-cultivating-a-culture-of-information-security> (Accessed: 14 November 2025).

Viegas, V. & Kuyucu, O. (2022) *IT Security Controls: A Guide to Corporate Standards and Frameworks*. Berkeley, CA: Apress.

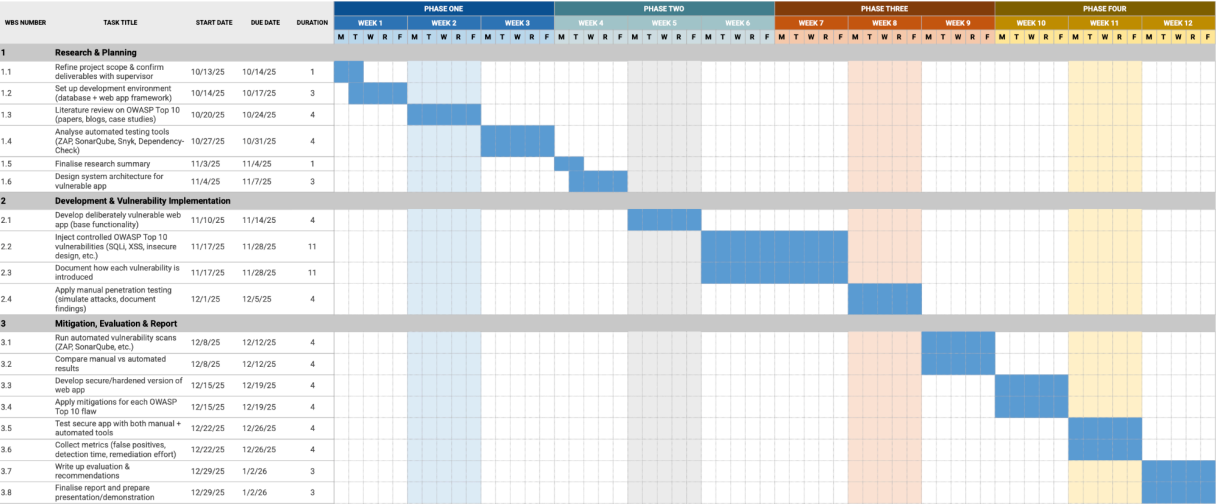
Wisse, E., Tedeschi, P., Sciancalepore, S. & Di Pietro, R. (2023) 'A2RID: Anonymous Direct Authentication and Remote Identification of Commercial Drones', *IEEE Internet of Things Journal*, 10(12), pp. 10587–10604. doi: 10.1109/JIOT.2023.3240477.

8. Appendices

8.1 Initial Gantt Chart

GANTT CHART

PROJECT TITLE	Detecting and Mitigating OWASP Top 10 Threats: Design, Implementation and Security Evaluation of a Database Driven Web App		
PROJECT SUPERVISOR	Alix Bergeret	DATE	10/6/25
PROJECT AUTHOR	Zubair Idris Aweda		



GANTT CHART

[illegible]